Diseño y Arquitectura de Software

Módulo 4 - Arquitectura

Facultad de
Ingeniería

VIGILADA MINEDUCACIÓN



Universidad de La Sabana

Universidad de La Sabana | Facultad de **Ingeniería**



Bienvenidos

En este módulo, exploraremos los estilos y patrones arquitectónicos, fundamentales en la construcción de sistemas de software sólidos y escalables. Aprenderemos a seleccionar el estilo y patrón de arquitectura más adecuado según los requerimientos no funcionales del proyecto, garantizando así el cumplimiento de estándares de rendimiento y seguridad.



Estilo arquitectónico



Estilo arquitectónico

Un estilo arquitectónico es un conjunto de principios, patrones y reglas que guían el diseño y la organización de un sistema de software.

Define la estructura general del sistema y establece las relaciones entre sus componentes principales.

Los estilos arquitectónicos proporcionan una manera estandarizada de abordar problemas comunes de diseño, permitiendo a los arquitectos de software tomar decisiones informadas y consistentes sobre cómo estructurar y construir un sistema.





Patrones Arquitectónicos

Los patrones arquitectónicos son soluciones probadas y eficientes para problemas recurrentes en el diseño de la arquitectura de software.

Se centran en la estructura general y la organización de un sistema de software, proporcionando un marco para la toma de decisiones arquitectónicas.

Características Principales:

- Establecen la distribución de responsabilidades entre los componentes del sistema.
- Definen la forma en que los componentes se comunican entre sí.
- Abordan aspectos como la escalabilidad, la seguridad y la disponibilidad del sistema.

Ejemplos Comunes:

- Arquitectura de Capas
- Modelo-Vista-Controlador (MVC)
- Arquitectura de Microservicios
- Arquitectura Hexagonal
- Arquitectura Orientada a Servicios (SOA)



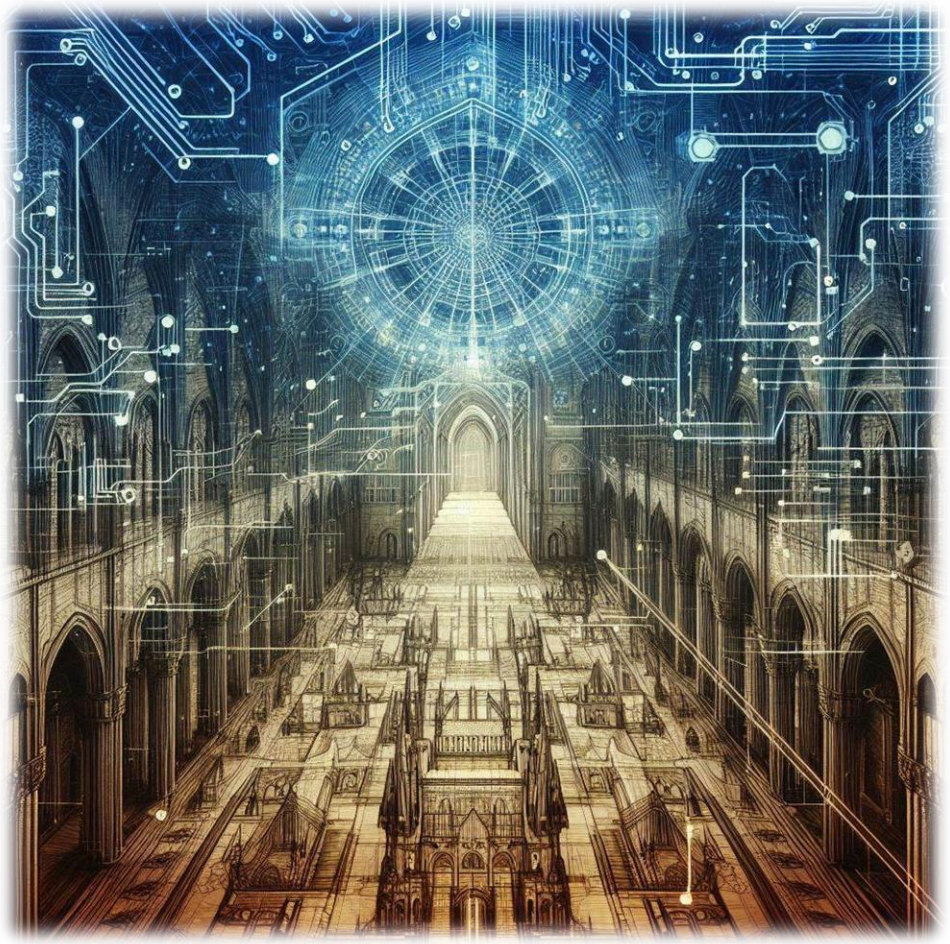
Facultad de Ingeniería

Universidad de La Sabana

Patrones Arquitectónicos

Un estilo arquitectónico es un conjunto de principios, patrones y reglas que guían el diseño y la organización de un sistema de software.

Define la estructura general del sistema y establece las relaciones entre sus componentes principales.



Importancia

Proporcionan un marco para abordar desafíos complejos de diseño de sistemas de software.

Permiten a los arquitectos de software tomar decisiones informadas y consistentes sobre la estructura del sistema.

Mejoran la calidad, la flexibilidad y la mantenibilidad del sistema a lo largo del tiempo.



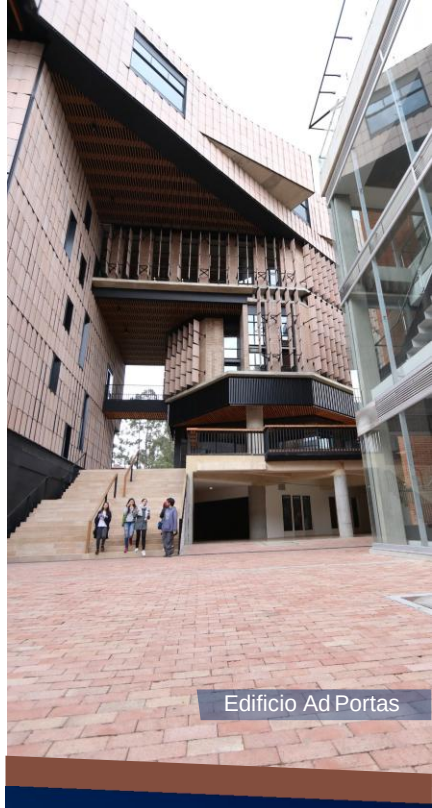
¿Qué es un patrón?

¿Qué son?

Estos patrones representan un enfoque estandarizado y comúnmente aceptado para abordar desafíos específicos relacionados con la organización, la estructura y la interacción de los componentes de un sistema de software.

Principios

Un patrón en arquitectura proporciona un esquema reutilizable y adaptable que los arquitectos de software pueden aplicar en diferentes contextos para resolver problemas similares.



Facultad de
Ingeniería

Universidad de
La Sabana

Diferencias entre patrones

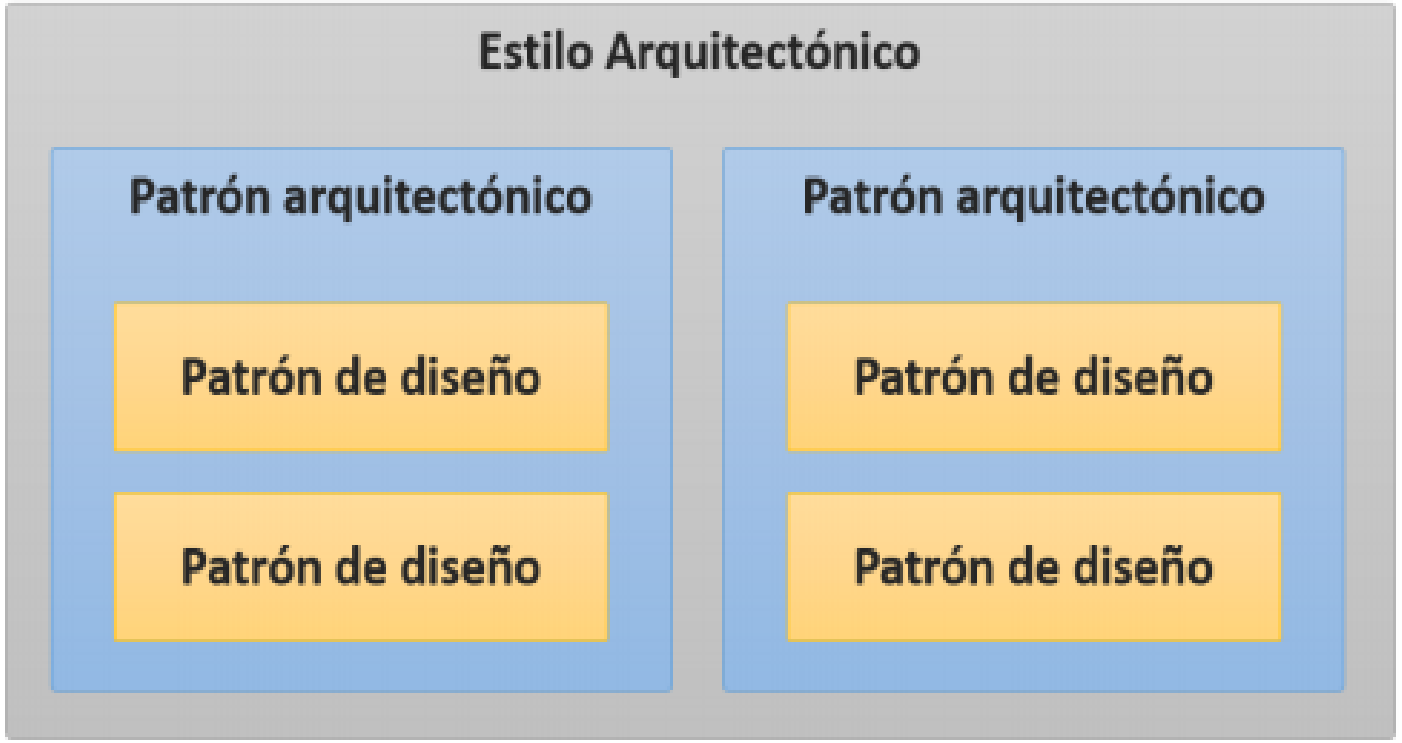


Fig 5: Relación entre patrones de diseño, patrones arquitectónicos y estilos arquitectónicos



Facultad de
Ingeniería

Universidad de
La Sabana

Tipos de patrones

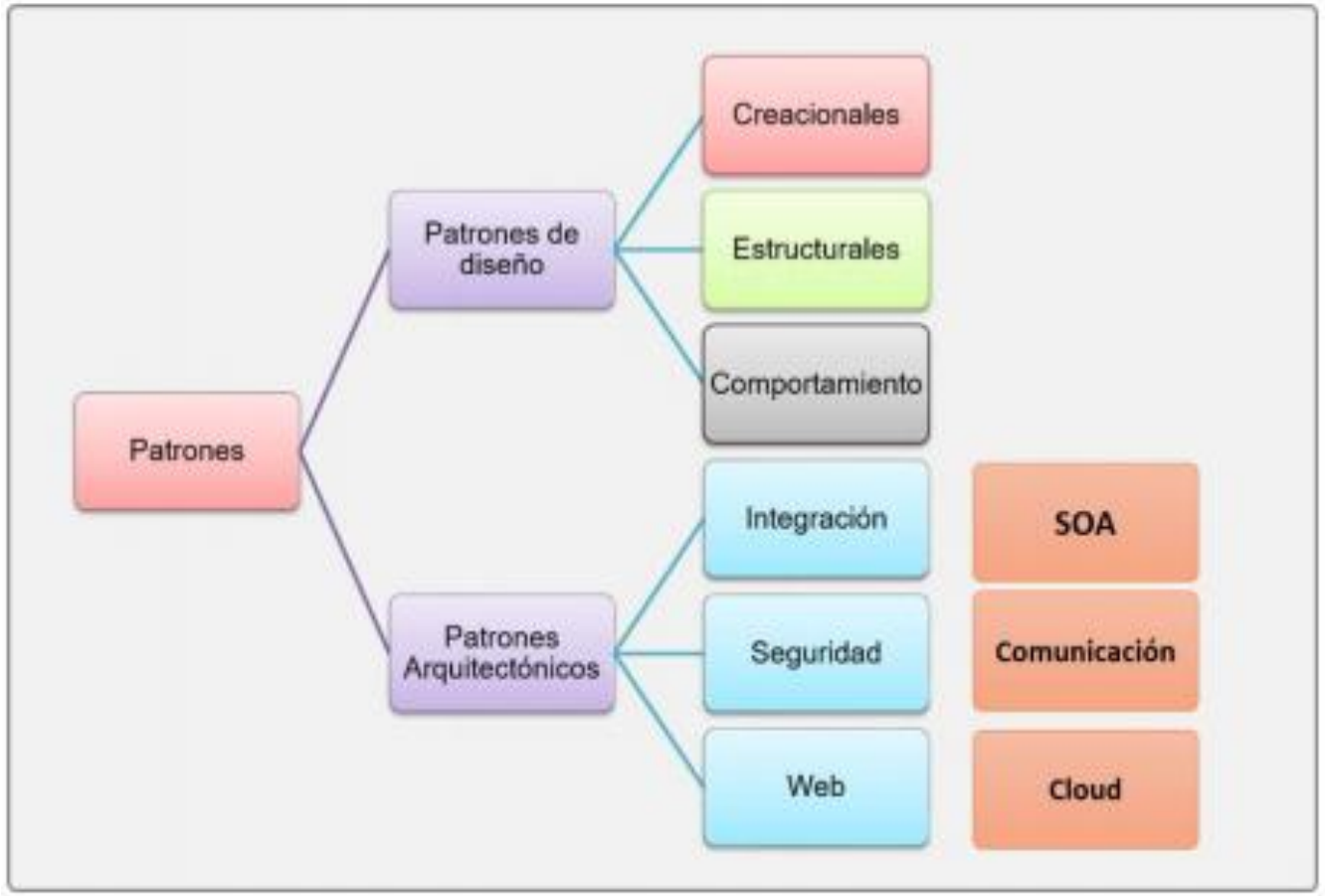
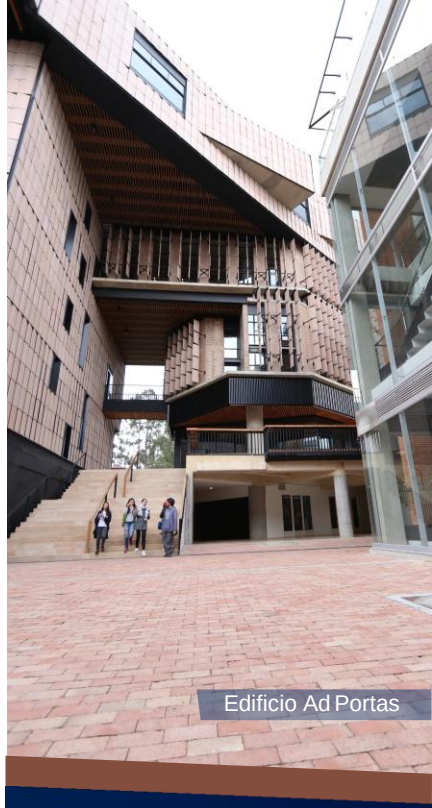


Fig 3: Tipos de patrones



Facultad de
Ingeniería

Universidad de
La Sabana

Los estilos arquitectónicos no determinan la tecnología ni la implementación de un sistema.

Un estilo arquitectónico es un conjunto de restricciones arquitectónicas colaborativas que restringen los roles y funciones de los elementos arquitectónicos, y entre elementos que pueden existir en cualquier arquitectura que siga ese estilo.

“Los estilos de la arquitectura es una restricción”





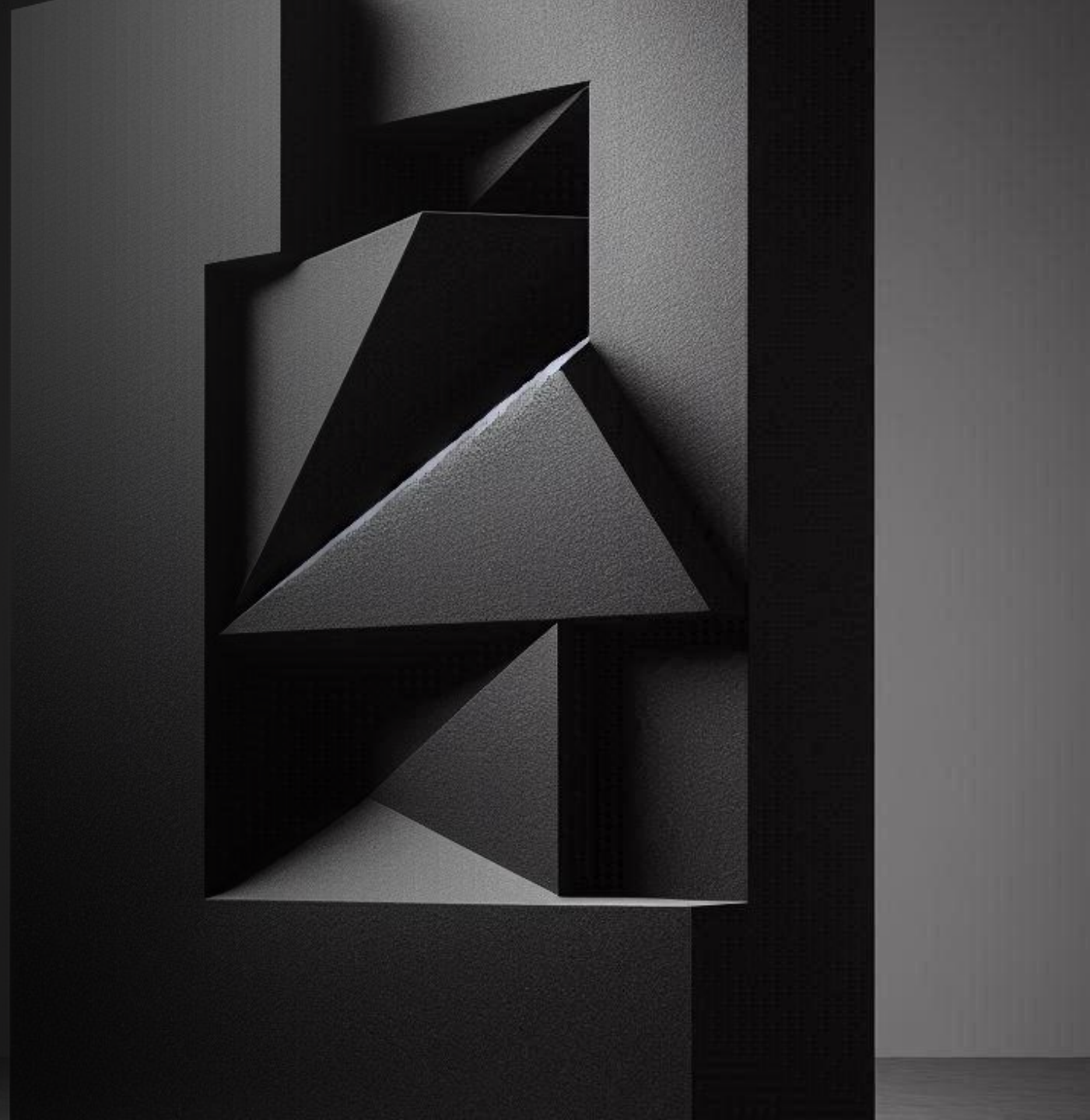
Los patrones de arquitectura describen la relación entre un conjunto de componentes para resolver un problema arquitectónico común en un contexto particular

¡El patrón arquitectónico es una aplicación específica de un estilo arquitectónico en un dominio de problema específico!



Estilos Arquitectónicos

Estilo Monolito





Estilo monolito

El **estilo arquitectónico monolítico** consiste en crear una aplicación autosuficiente que contiene **toda la funcionalidad necesaria** para realizar la tarea para la cual fue diseñada, **sin depender de componentes externos**.

una aplicación monolítica es **una unidad cohesiva de código**. Aunque las arquitecturas más modernas han evolucionado hacia enfoques más modulares, las aplicaciones monolíticas siguen siendo relevantes en ciertos escenarios.

- Son aplicaciones autosuficientes (no requieren de nada para funcionar).
- Realizan de punta a punta todas las operaciones para terminar una tarea.
- Tienden a ser aplicaciones grandes, aunque no es un requisito.
- Cada instalación administra su propia base de datos.
- Todo el sistema corre sobre una sola plataforma.
- Es fácil perder el rumbo por malas prácticas de programación, y conocer un proyecto avanzado puede ser abrumador



Estructura

La **estructura monolítica** en arquitectura de software se caracteriza por tener **todos los componentes** de una aplicación integrados en un solo bloque

Unidad única / empaquetado

Componentes acoplados

Escalabilidad vertical / CPU o Memoria

Despliegue único / Ejecutable único

Facilidad de desarrollo inicia

Desafíos a largo plazo

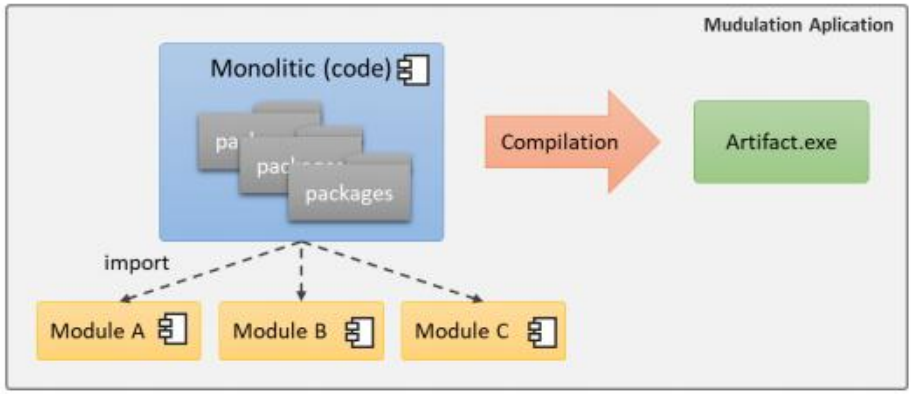


Fig 34 – Proceso de compilación.

Si los módulos son compilados por separado y son instalados como complementos **ya no se trata de un Monolítico**, sino que pasa a ser un estilo arquitectónico de Microkernel.

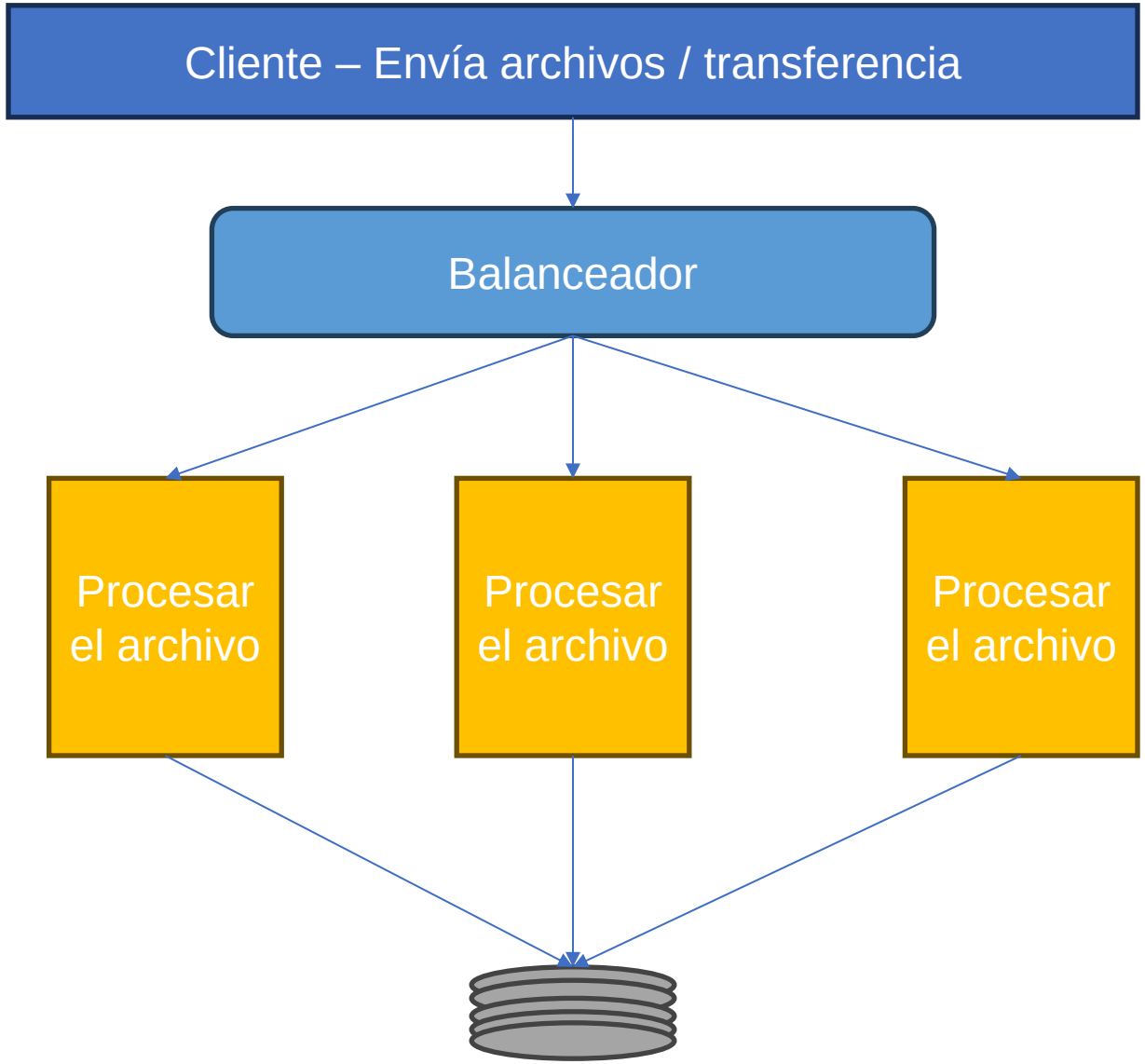


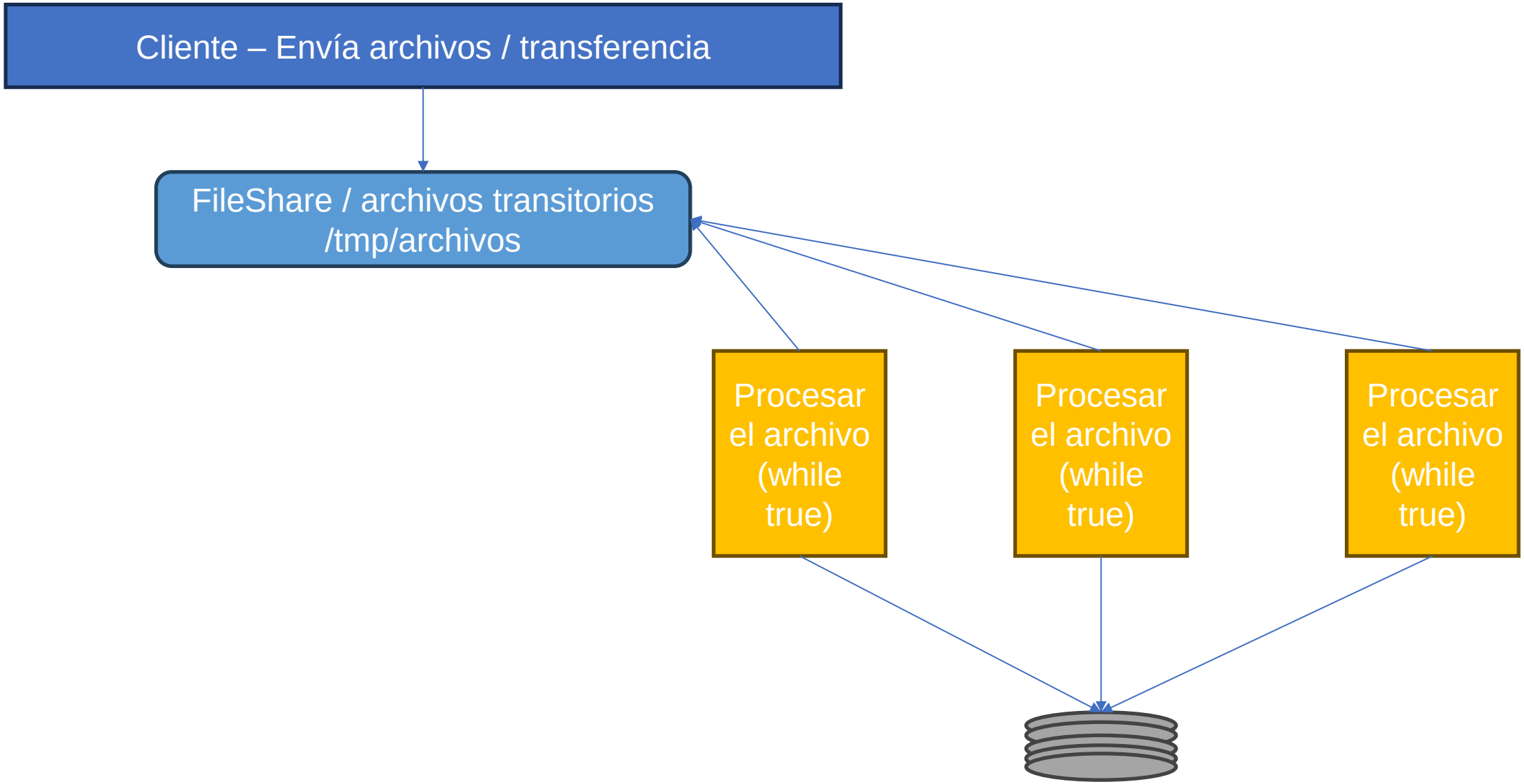
Facultad de Ingeniería

Universidad de La Sabana

Ventajas y Desventajas

Característica	Ventajas	Desventajas
Debugabilidad	- Fácil de depurar, ya que todo el código está en un solo lugar.	- Difícil de aislar errores específicos debido al acoplamiento.
Desempeño	- Rendimiento inicialmente bueno debido a la falta de comunicación entre componentes.	- Dificultad para escalar horizontalmente (agregar más servidores).
Escalabilidad	- Escalabilidad vertical (aumento de recursos en el mismo servidor).	- Difícil de escalar horizontalmente (agregar más servidores).
Mantenibilidad modular	- Facilidad de desarrollo inicial.	- A medida que crece, puede volverse difícil de mantener.
Facilidad de implementación	- Implementación sencilla como una sola unidad.	- Actualizaciones requieren volver a implementar todo.
Portabilidad	- No hay dependencias externas, lo que facilita la portabilidad.	- Puede estar atado a una tecnología específica.
Testeabilidad	- Pruebas unitarias más sencillas, ya que no hay comunicación entre componentes.	- Pruebas de integración más difíciles debido al acoplamiento.
Tolerancia a fallos	- Si un componente falla, toda la aplicación puede verse afectada.	- Mayor riesgo de fallos debido al acoplamiento.


Cliente – Envía archivos / transferencia
Balanceador
Procesar el archivo
Procesar el archivo
Procesar el archivo


Cliente – Envía archivos / transferencia
FileShare / archivos transitorios
/tmp/archivos
Procesar el archivo (while true)
Procesar el archivo (while true)
Procesar el archivo (while true)

¿Cuándo usar un monolito?

Ventajas de un Monolito:

1. **Velocidad de desarrollo**: Al principio de un proyecto, los monolitos son más rápidos de desarrollar debido a su simplicidad. Toda la aplicación se basa en una única base de código.
2. **Implementación sencilla**: Publicar cambios en un monolito es más fácil, ya que todo está en una sola unidad.

Desventajas de un Monolito:

1. **Actualizaciones restrictivas**: Para hacer cambios, debes actualizar toda la pila, lo que puede ser lento y restrictivo.
2. **Dificultad de mantenimiento**: A medida que crece, mantener un monolito puede volverse complicado.



¿Cuándo usar un monolito?

1. Al inicio de un proyecto para simplificar la gestión de código y la implementación.
2. Cuando la sobrecarga cognitiva no es un problema y la velocidad de desarrollo es crucial.
3. Cuando la aplicación va **creciendo**, se puede migrar a arquitecturas más modulares.
4. Utilizar un estilo Monolítico **no es malo**, lo malo sería quedarnos siempre en este estilo y no ir migrando a otros estilos más sofisticados cuando la aplicación **va creciendo**.



¿Cuándo dejar de usar un monolito?

La **transición de un monolito a una arquitectura más modular** es una decisión importante en el desarrollo de software.

- **Complejidad creciente**: Si la aplicación monolítica se ha vuelto difícil de mantener debido a su tamaño y complejidad, considera dividirla en componentes más pequeños.
- **Dificultad para escalar**: Los monolitos pueden ser difíciles de escalar horizontalmente. Si necesitas manejar más carga y escalabilidad, una arquitectura basada en microservicios podría ser más adecuada.
- **Necesidad de implementar nuevas tecnologías**: Si deseas adoptar nuevas tecnologías o lenguajes de programación, un monolito puede ser restrictivo. Las arquitecturas más modulares permiten una mayor flexibilidad.
- **Equipo diverso**: Si trabajas con un equipo grande y diverso, los microservicios pueden facilitar la colaboración al permitir que diferentes equipos se enfoquen en componentes específicos.
- **Mayor agilidad**: Las arquitecturas basadas en microservicios permiten implementaciones más rápidas y actualizaciones independientes. Si buscas mayor agilidad, considera la transición.





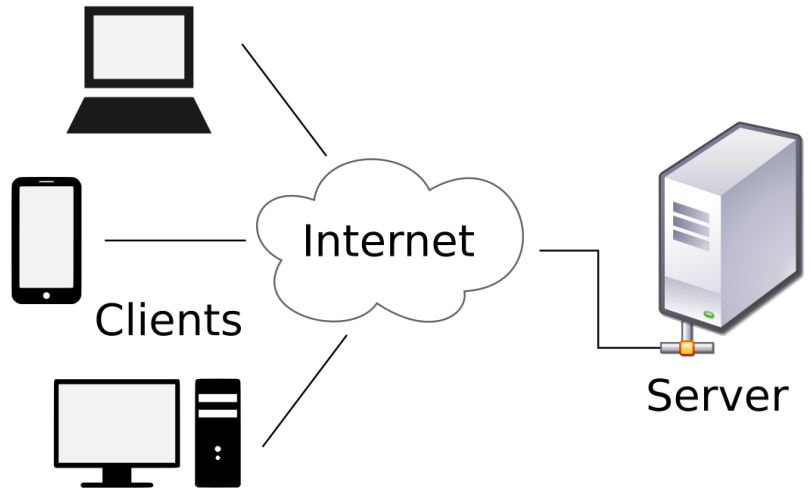
Cliente - Servidor



Cliente - Servidor

La arquitectura cliente-servidor es un modelo ampliamente utilizado en sistemas informáticos y redes. Se compone de dos partes claramente diferenciadas: el **servidor** (que almacena datos y proporciona servicios) y el **cliente** (que solicita servicios al servidor).

- **Comunicación**: Los clientes envían solicitudes al servidor, y este responde con los datos o servicios requeridos.
- **Protocolos**: Utiliza protocolos de comunicación estándar, como HTTP, TCP/IP o RPC.
- **Escalabilidad**: Permite agregar más servidores para manejar la carga creciente.



Estructura

Componentes:

1. **Cliente**: Es la parte que solicita servicios o recursos al servidor.
2. **Servidor**: Proporciona servicios o recursos a los clientes.
3. **Librería con objetos comunes**: A veces, se utiliza una librería compartida para objetos o funciones que ambos, cliente y servidor, necesitan. Como Clases base, Objetos de entidad, DTO o interfaces (**DRY**).

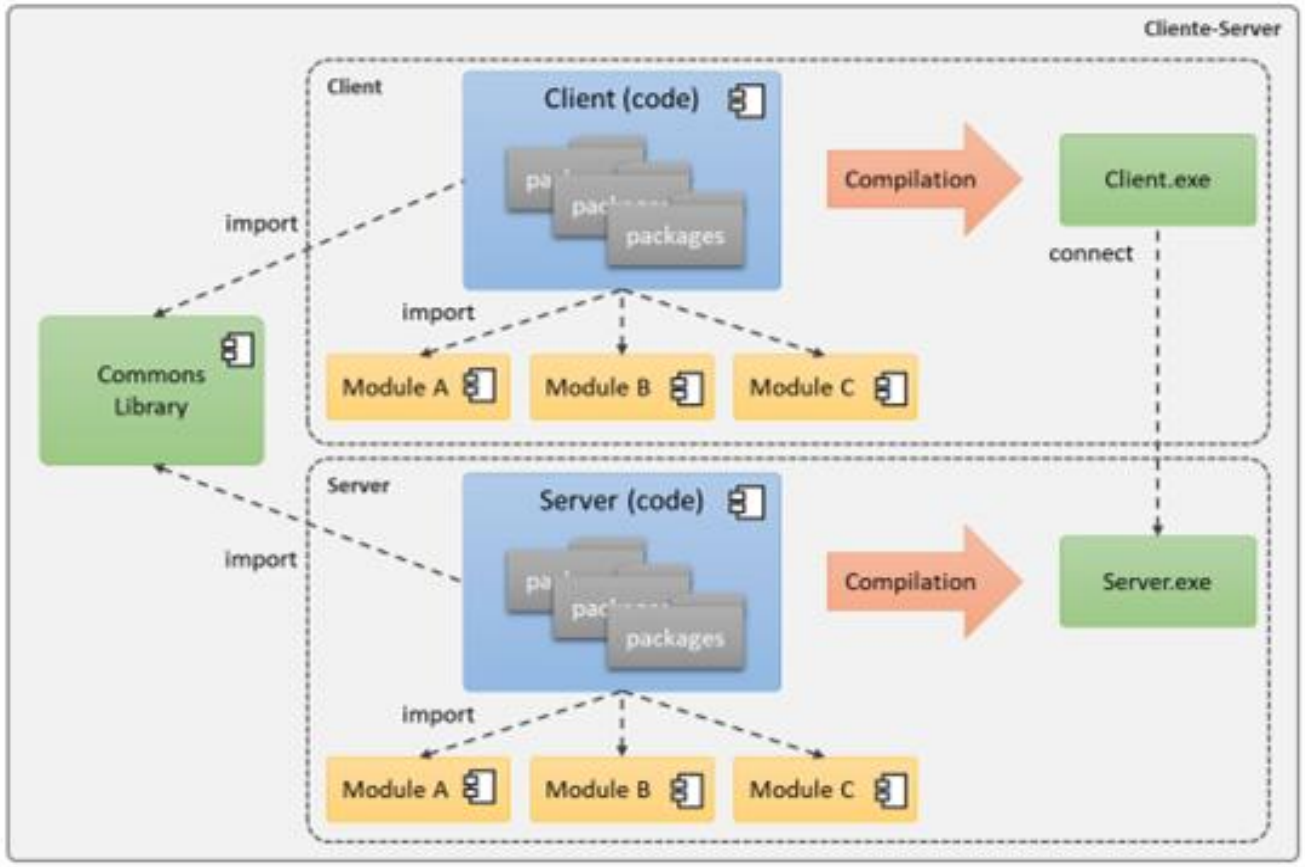


Fig 37 - Estructura de una aplicación Cliente-Servidor.





Caracteristicas

Cliente:

1. Funciona como una **capa de presentación de datos**.
2. Define acciones para **modificar el estado del servidor**.

Servidor:

1. Realiza **el trabajo pesado**.
2. **Expone sus servicios** para permitir una comunicación continua y bidireccional.

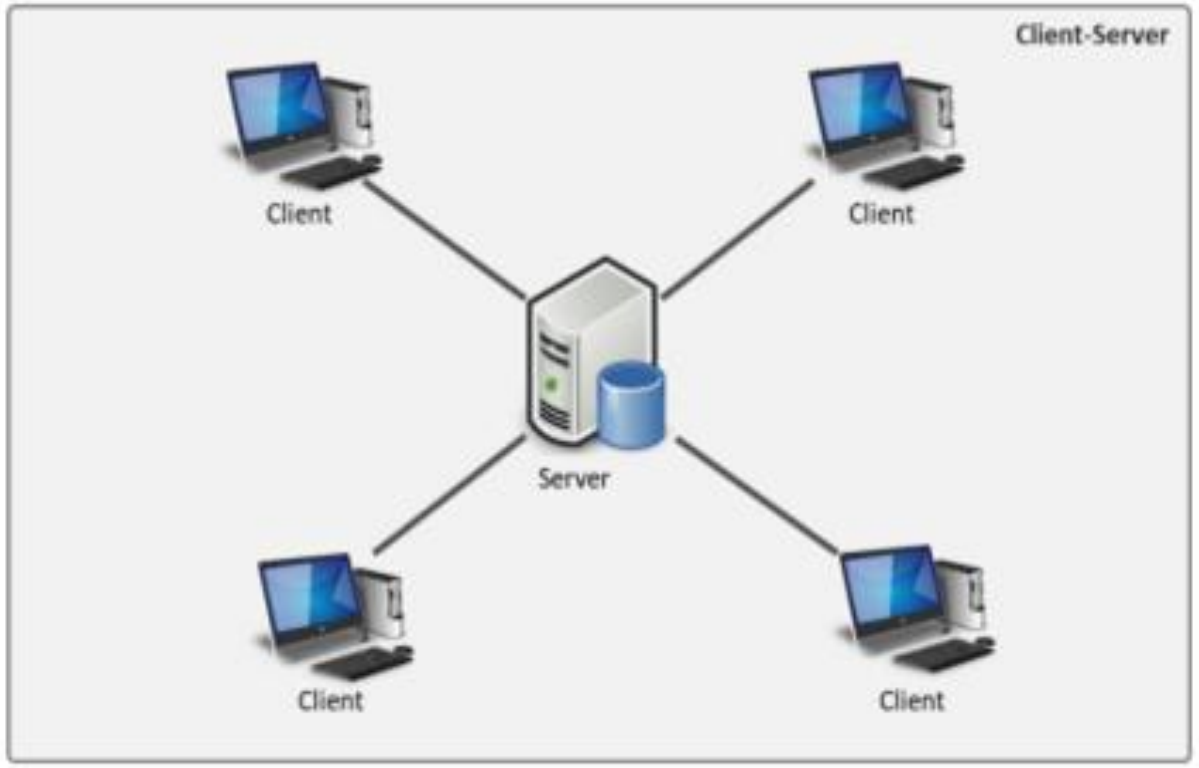


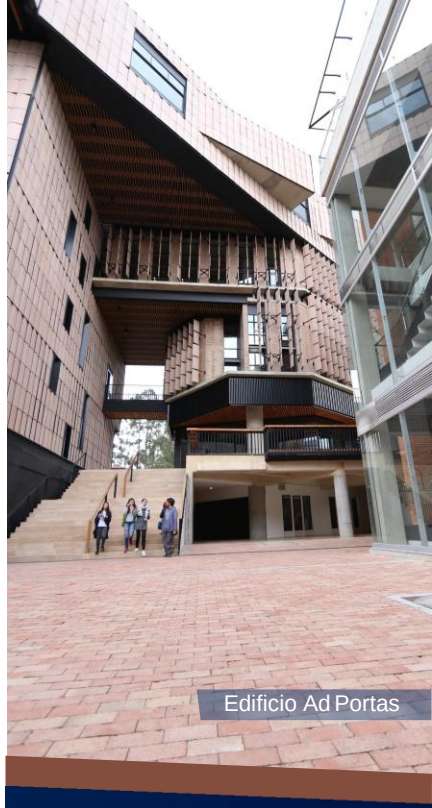
Fig 35 - Arquitectura Cliente-Servidor

Características

- **Interdependencia Cliente-Servidor**:
  - En esta arquitectura, **el cliente y el servidor se necesitan mutuamente**.
  - El **cliente no es útil sin un servidor disponible**, y el **servidor carece de propósito sin un cliente que lo utilice**.

- **Arquitectura Distribuida**:
  - Se considera una **arquitectura distribuida**, ya que sus partes están distribuidas en diferentes equipos o ubicaciones físicas.

- **Desarrollo de Aplicaciones Diferentes**:
  - El **cliente y el servidor** se desarrollan como **aplicaciones separadas**, pero **siguen el mismo protocolo de comunicación**.



Facultad de **Ingeniería**

Universidad de La Sabana

Características

- **Rol del Servidor**:
  - El **servidor** es el único que **accede a los datos**, los procesa y los sirve exclusivamente a los clientes.

- **Función del Cliente**:
  - El **cliente** se instala en dispositivos con **recursos limitados** y **actúa como un visor de los datos**.
  - **Delega las operaciones pesadas al servidor** para mantener su eficiencia.

- **Relación Simbiótica**:
  - Aunque tanto el cliente como el servidor son como **aplicaciones monolíticas**, **no son autosuficientes**.
  - **Trabajan en conjunto**, creando una **relación simbiótica** para proporcionar servicios a los usuarios.





Facultad de Ingeniería

Universidad de La Sabana

Ventajas y Desventajas

Característica	Ventajas	Desventajas
Centralización	- Mejora la consistencia de los datos y la eficiencia operativa.	- Puede crear un punto único de falla y aumentar los costos de infraestructura.
Confiabilidad	- Asegura un rendimiento constante y predecible.	- Puede ser costoso de implementar y mantener.
Facilidad de distribución - Cliente	- Permite una rápida implementación y actualizaciones sencillas.	- Puede requerir compatibilidad específica del sistema cliente.
Instalabilidad - Cliente	- Facilita la adopción por parte del usuario final.	- Puede presentar problemas con sistemas operativos o hardware específicos.
Separación de Responsabilidades	- Mejora la organización y permite una especialización más profunda.	- Puede complicar la comunicación entre departamentos o funciones.
Seguridad	- Protege contra amenazas externas e internas.	- Requiere actualizaciones constantes para abordar nuevas amenazas.
Testabilidad - Cliente	- Facilita el diagnóstico y la resolución de problemas.	- Puede ser limitada por las capacidades del cliente o el entorno del usuario final.
Tolerancia a fallos - Servidor	- Asegura la continuidad del servicio ante fallos individuales.	- Puede ser costosa e incrementar la complejidad del sistema.

¿Cuándo usar el estilo cliente-servidor?

- **Aplicaciones en Tiempo Real**:
  - Estas aplicaciones requieren **reflejar los datos al instante**.
  - Tecnologías como **Sockets y Web Sockets** permiten una **comunicación bidireccional continúa** basada en protocolos TCP/IP.

- **Aplicaciones Centralizadas de Alto Rendimiento**:
  - Estas aplicaciones necesitan **tener un solo origen confiable de datos**.
  - También pueden requerir **operaciones con alto consumo de recursos**.



¿Cuándo dejar de usar el estilo cliente-servidor?

La **arquitectura cliente-servidor** es ampliamente utilizada, pero hay momentos en los que podría ser apropiado dejar de usarla.

- **Vulnerabilidad ante ataques externos**:
  - Al estar todos los recursos concentrados en un solo lugar, puede ser más fácil para los hackers atacarlos todos a la vez.
- **Dependencia del ancho de banda**:
  - Si el ancho de banda es bajo o inestable, el rendimiento general del sistema será menor.
- **Cambios en los requisitos de seguridad**:
  - Si los requisitos de seguridad cambian y necesitas una mayor protección contra amenazas, podrías evaluar otras arquitecturas más seguras.
- **Nuevas tecnologías y paradigmas**:
  - Si surgen nuevas tecnologías o paradigmas que ofrecen ventajas significativas en términos de rendimiento, escalabilidad o seguridad, podrías considerar una transición.
- **Necesidad de escalabilidad extrema**:
  - Si tu aplicación requiere una escalabilidad extrema y una distribución global, podrías considerar arquitecturas más descentralizadas, como microservicios o computación en la nube



Per – to – Per

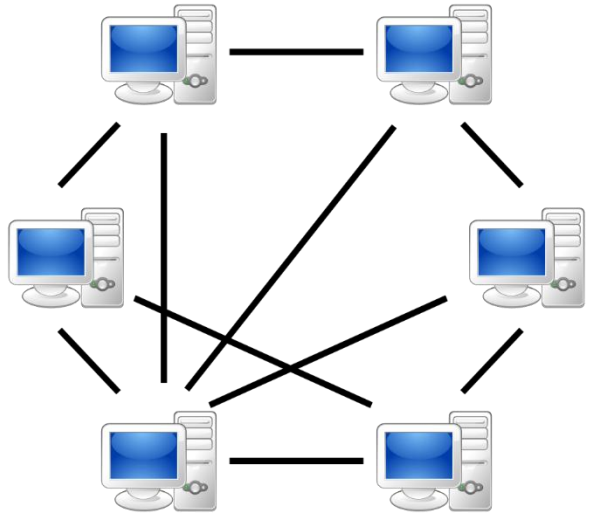




Per – to - Per

La **arquitectura Peer-to-Peer (P2P)** es un estilo arquitectónico en el que cada nodo o “par” puede actuar tanto como servidor como cliente. En una red P2P, no es necesario un servidor central para administrar la comunicación entre los dispositivos conectados.

Todos los nodos pueden comunicarse directamente entre sí



Estructura y características

- **Descentralización**:
  - A diferencia de la arquitectura cliente-servidor, donde hay un servidor central, en P2P no existe un punto único de control.
  - Cada nodo tiene igualdad de capacidades y responsabilidades.

- **Funcionamiento**:
  - Cada nodo puede **compartir recursos** (como archivos, datos o servicios) con otros nodos.
  - Los nodos pueden **descargar y subir** recursos entre sí.





Ejemplos

Intercambio de archivos:
Las redes P2P son comunes en aplicaciones de intercambio de archivos, donde los usuarios pueden compartir y descargar archivos directamente entre sí.

BitTorrent es un protocolo P2P que permite a los usuarios compartir y descargar archivos directamente entre sí sin depender de un servidor central. Los usuarios descargan fragmentos de archivos de múltiples fuentes simultáneamente.

Redes descentralizadas:
Algunas criptomonedas utilizan una arquitectura P2P para validar transacciones sin depender de un servidor central.

Bitcoin es una criptomoneda que opera en una red P2P. Los nodos (mineros) validan y registran las transacciones en un libro de contabilidad descentralizado llamado blockchain.



Ejemplos

Comunicación directa entre dispositivos:
Cuando se requiere una **comunicación simétrica** entre nodos, como en aplicaciones de **mensajería instantánea** o **llamadas de voz**.

Skype, que permite llamadas y mensajes entre usuarios sin intermediarios.

Distribución de carga y escalabilidad:
En sistemas donde se necesita **distribuir la carga** entre los nodos para mejorar el rendimiento y la escalabilidad.

Redes de distribución de contenido (CDN), donde los nodos locales almacenan y entregan contenido a los usuarios cercanos.

Ventajas y Desventajas

Ventajas	Desventajas
Descentralización y escalabilidad	- **Estabilidad y rendimiento de la red**: La falta de un servidor central puede afectar la estabilidad y el rendimiento de la red.
Intercambio eficiente de recursos	- **Preocupaciones legales y de seguridad**: Al compartir recursos directamente, pueden surgir problemas legales y de seguridad.
Redundancia y tolerancia a fallos	- **Falta de gestión centralizada**: La ausencia de un control central dificulta la administración y el mantenimiento de la red.
Privacidad y seguridad mejoradas	- **Consumo de ancho de banda**: Las redes P2P pueden generar un alto consumo de ancho de banda, especialmente en aplicaciones de intercambio de archivos.
	- **Vulnerabilidades de seguridad**: Al no tener un punto central de protección, las redes P2P pueden ser vulnerables a ataques y amenazas.



Facultad de
Ingeniería

Universidad de
La Sabana

Arquitectura en capas





Arquitectura en capas

La **arquitectura en capas** es un modelo de diseño de software que organiza una aplicación en diferentes niveles o capas, cada una con un rol específico y bien definido. En este enfoque, las funcionalidades del sistema se separan en módulos lógicos, y cada capa se comunica con las adyacentes mediante interfaces claras.

Capas:

- Cada capa es un conjunto de componentes o módulos de software que realizan una función específica.
- Las capas se organizan jerárquicamente, desde la más baja (encargada de detalles de implementación) hasta la más alta (que proporciona la interfaz de usuario y la lógica de presentación).
- Capas comunes incluyen la **capa de presentación**, la **capa de lógica de negocio**, la **capa de acceso a datos** y la **capa de almacenamiento (al menos 3)**.

Interfaces:

- Las interfaces definen cómo las capas interactúan entre sí.
- Proporcionan un conjunto de operaciones que una capa puede realizar o solicitar a otra.
- Las interfaces son esenciales para determinar la comunicación y la separación de responsabilidades entre las capas.

Características

Organización en Capas:

- Divide la aplicación en **capas**, asignando a cada una un **rol específico y bien definido**.
- El estilo arquitectónico no dicta un número específico de capas, sino que se enfoca en que cada capa tenga una **responsabilidad única** (SoC).

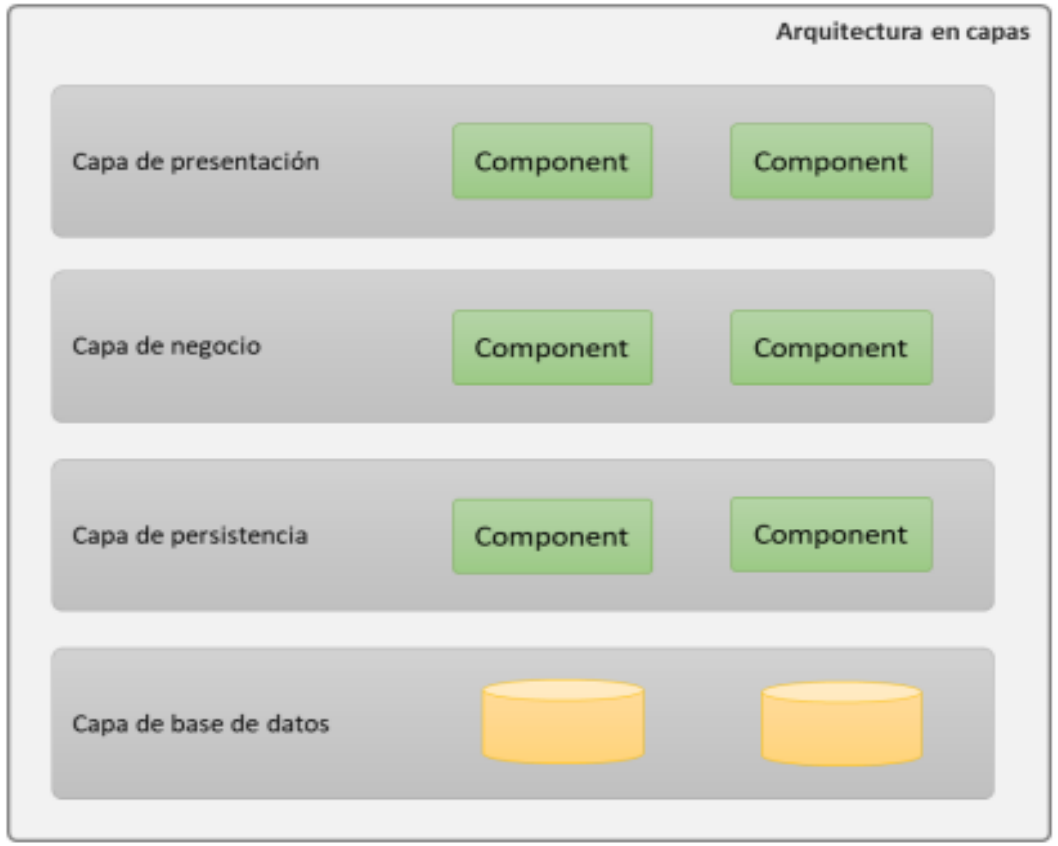


Fig 48 - Arquitectura en capas





Características

Capas Comunes:

- A menudo, se implementan **cuatro capas** (presentación, negocio, persistencia y base de datos).
- Sin embargo, es **habitual combinar** la capa de negocio y la de persistencia, lo cual puede considerarse un **antipatrón** si no se maneja adecuadamente.



Fig 48 - Arquitectura en capas

Estructura

Organización en Capas:

1. Las capas se **disponen horizontalmente**, de modo que cada capa solo puede comunicarse con la capa inmediatamente inferior (principio de **aislamiento de capas**).
2. Cada capa se diseñó originalmente como un **componente independiente**, lo que permite su **despliegue por separado**.

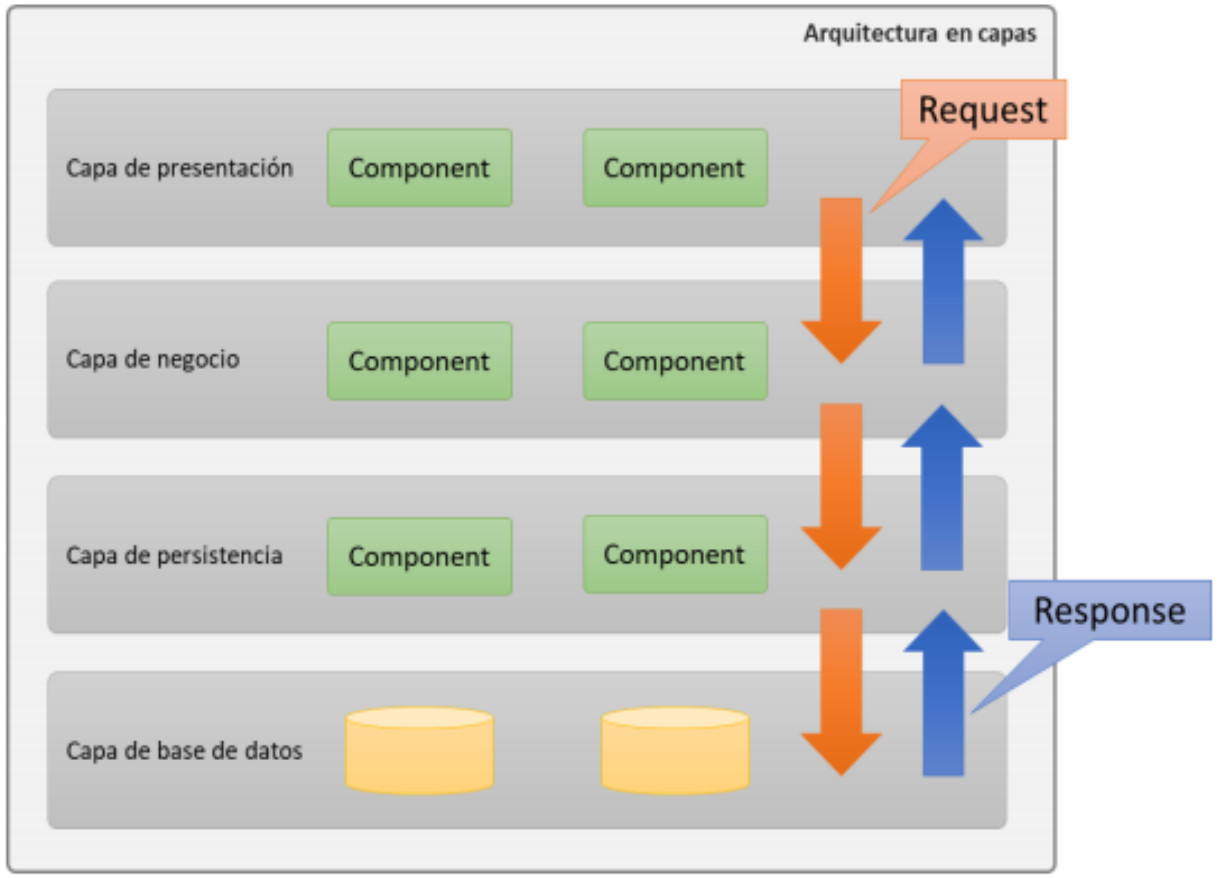


Fig 49 - Comunicación entre capas.





Estructura

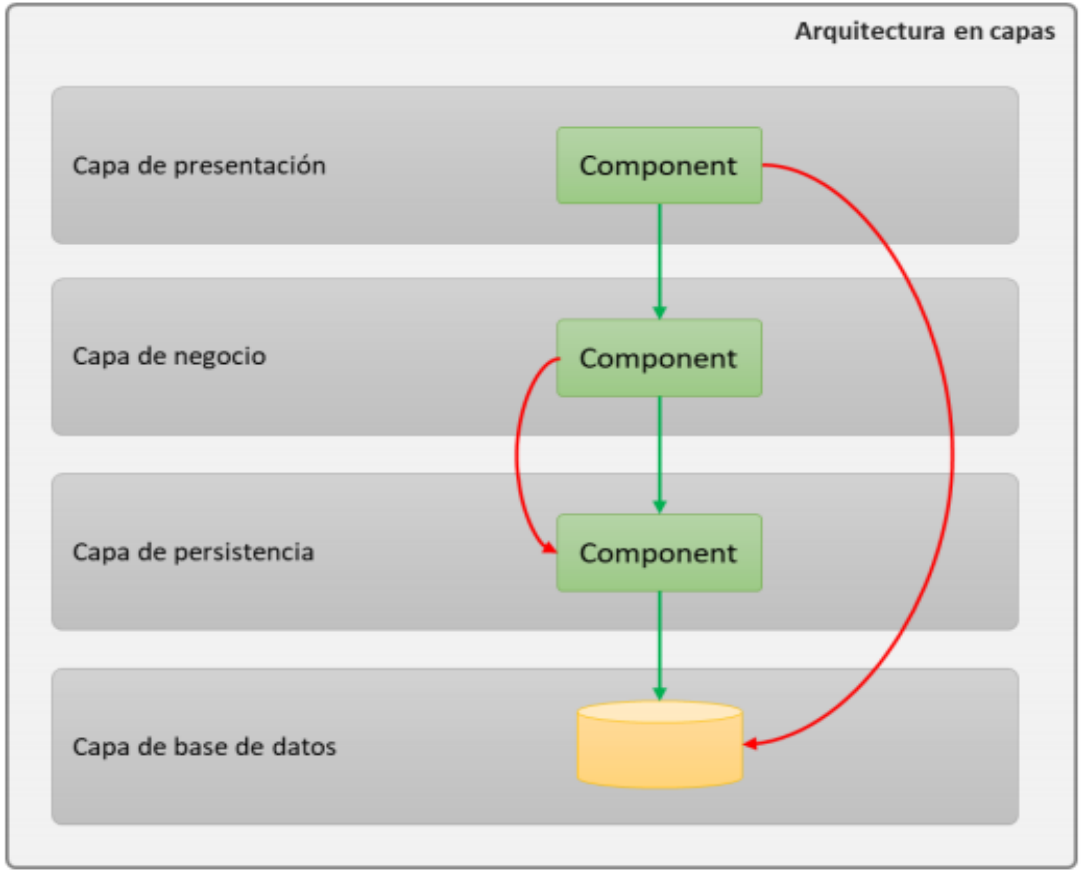


Fig 50 - Salto indebido entre capas.

Flujo de Comunicación en Capas:

- **Evitar saltarse una capa** y comunicarse directamente con una capa inferior suele ser un **grave error**.
- Esto puede generar **desorden** y dificultar la comprensión del flujo de datos (conocido como el **antipatrón spaghetti**).



Ejemplos

Ejecución en una Arquitectura de 3 Capas: El diagrama muestra cómo se ejecuta una **consulta de una nueva página web** en una arquitectura de **3 capas**.

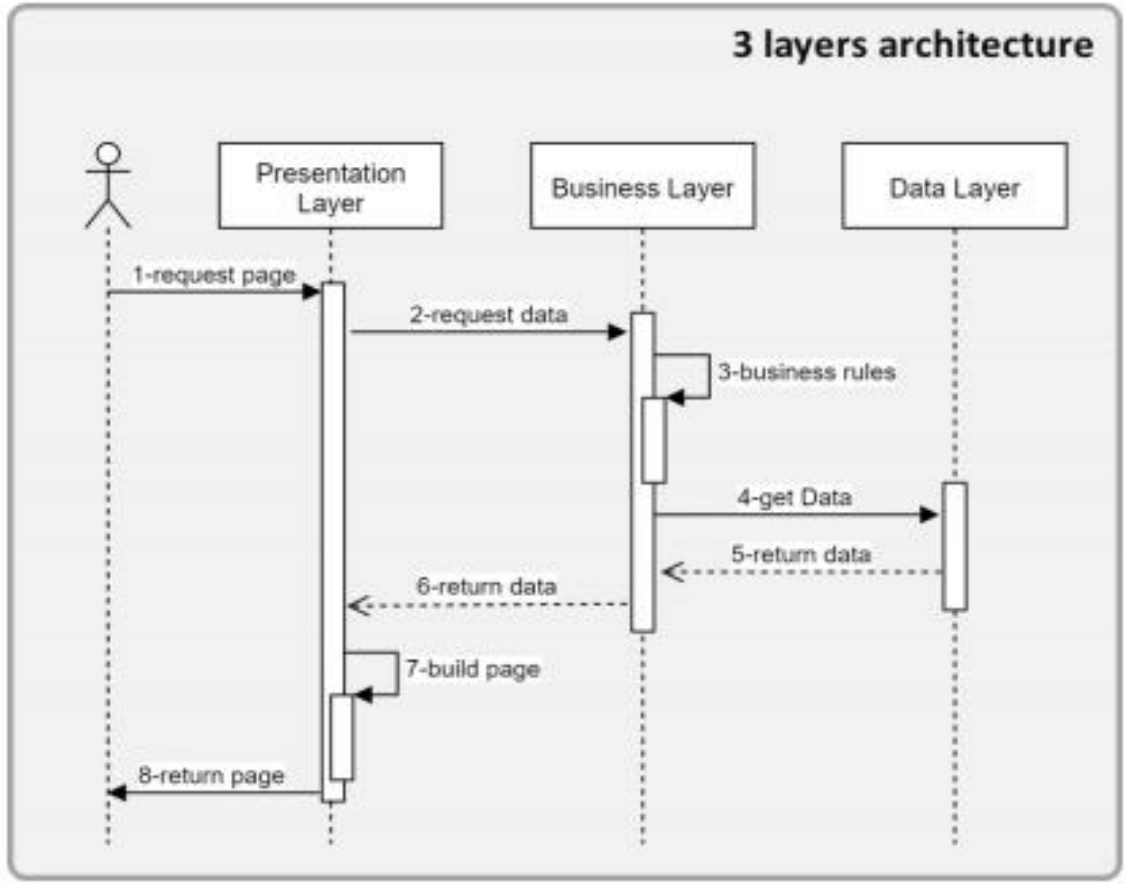


Fig 54 - Diagrama de secuencia de la ejecución en una arquitectura de 3 capas.



Facultad de Ingeniería

Universidad de La Sabana

Ventajas y Desventajas

Factor	Ventajas	Desventajas
Facilidad de despliegue	Permite una instalación y actualización sencilla y rápida.	Puede ser complejo si las capas no están bien definidas.
Escalabilidad	Facilita la expansión para soportar más usuarios o características.	Puede ser limitada por la dependencia entre capas.
Facilidad de implementación	Simplifica el desarrollo al dividir en módulos manejables.	La comunicación entre capas puede complicar la implementación.
Mantenibilidad	Las actualizaciones y correcciones son más fáciles con módulos separados.	La interdependencia puede complicar el mantenimiento.
Portabilidad	Permite mover o reutilizar capas en diferentes sistemas o plataformas.	La compatibilidad entre capas puede ser un desafío.
Rendimiento	Optimización específica para cada capa mejora el rendimiento general.	La comunicación entre capas puede afectar el rendimiento.
Seguridad	Permite enfocar medidas de seguridad específicas por capa.	Las vulnerabilidades pueden surgir en las interfaces entre capas.
Separación de responsabilidades	Clarifica roles y responsabilidades, facilitando la gestión del proyecto.	Puede llevar a una sobrecarga administrativa si no se maneja bien.
Testeabilidad	Facilita las pruebas al aislar las capas para pruebas individuales.	Las interacciones complejas entre capas pueden complicar las pruebas.
Tolerancia a fallos	Permite diseñar estrategias de recuperación ante fallos en capas.	La propagación de fallos entre capas puede ser un desafío.

¿Cuándo usar el estilo arquitectura por capas?

La arquitectura por capas es a menudo el **estilo predeterminado** cuando un arquitecto no está seguro de qué estilo utilizar.

En **entornos empresariales**, es apropiado para sistemas comunes como ventas, inventarios, pedidos o aplicaciones web empresariales.

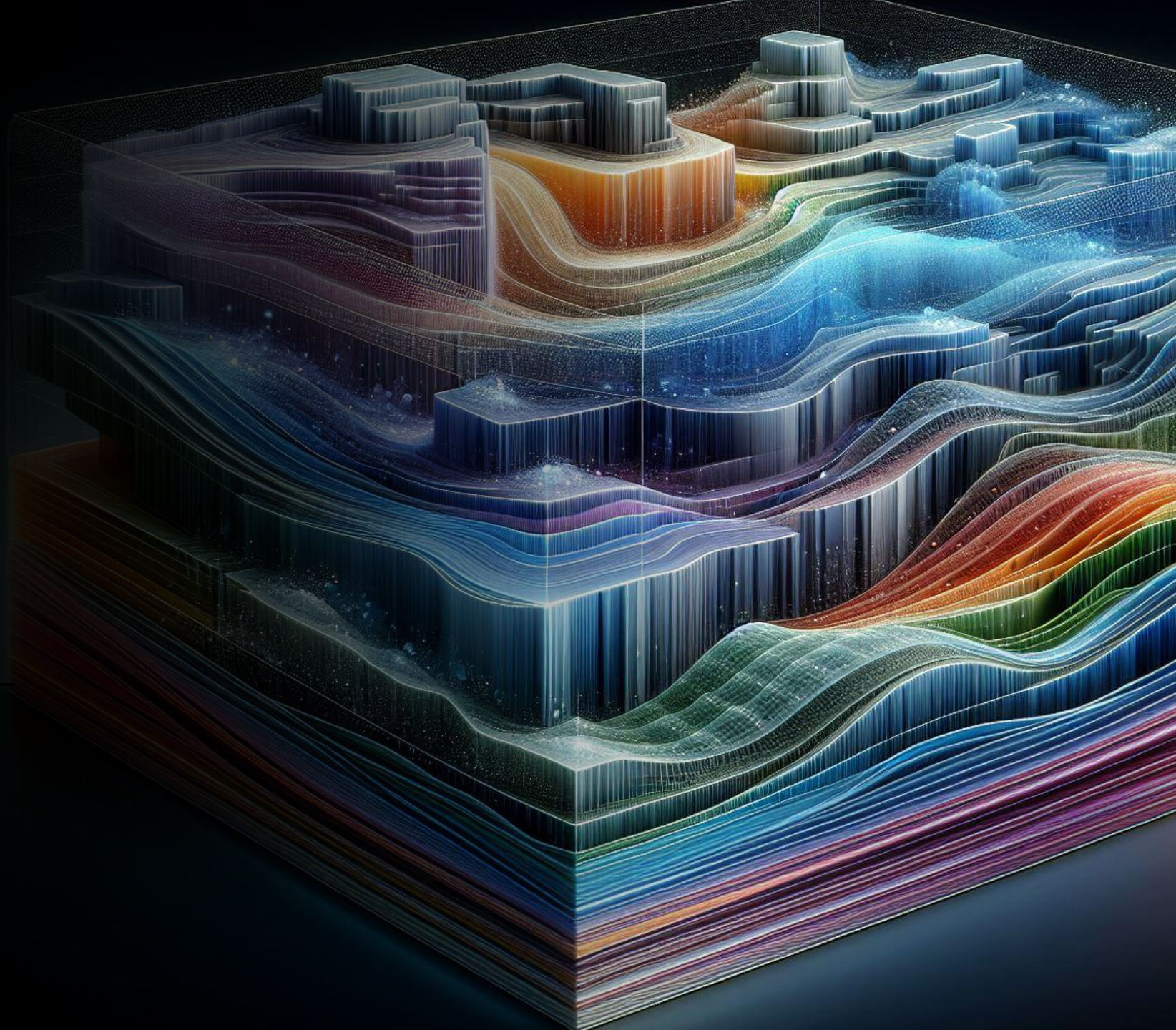
Estas aplicaciones, que a menudo evolucionan desde una arquitectura monolítica, suelen ser de **uso interno** y tienen **poca proyección de escalamiento**.

En términos generales, las capas en **aplicaciones web** incluyen:

- **Front-end**: Interfaz de usuario visible para los usuarios.
- **API REST**: Capa de lógica de negocio y acceso a datos.
- **Base de datos**: Almacena la información persistente.



N-Tiers





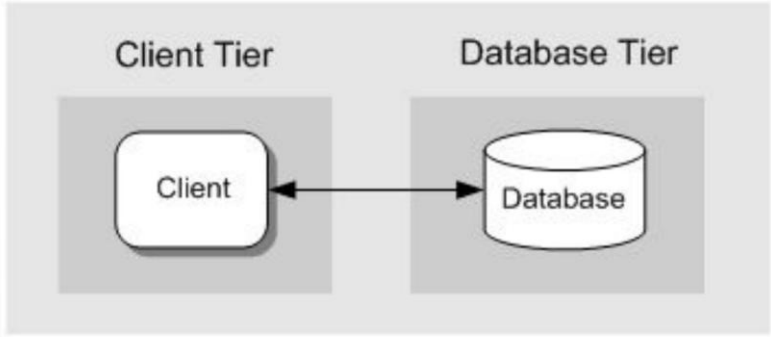
N-Tiers

La **arquitectura N-Tiers** (también conocida como arquitectura multinivel) es un enfoque común para organizar aplicaciones. En este estilo, la aplicación se divide en capas lógicas y niveles físicos. Cada capa tiene una responsabilidad específica, y las capas se comunican entre sí de manera jerárquica.

Capas y Niveles:

- Las capas son una forma de **separar responsabilidades** y administrar dependencias.
- Los niveles están **físicamente separados** y se ejecutan en máquinas diferentes.
- Un nivel puede llamar a otro nivel directamente o usar patrones de **mensajería asincrónica** a través de una cola de mensajes.

The 2-tier deployment pattern



Características

Capa Cerrada

En la **capa cerrada**, una capa solo puede llamar a la inmediatamente inferior.

En una aplicación web, la **capa de presentación** solo puede llamar a la **capa de lógica de negocio**, y esta última solo puede comunicarse con la **capa de acceso a datos**.

vs.

Capa Abierta

En la **capa abierta**, una capa puede llamar a cualquiera de las capas inferiores.

En una aplicación, la **capa de presentación** puede llamar directamente a la **capa de acceso a datos** o incluso a la **capa de seguridad** si es necesario.

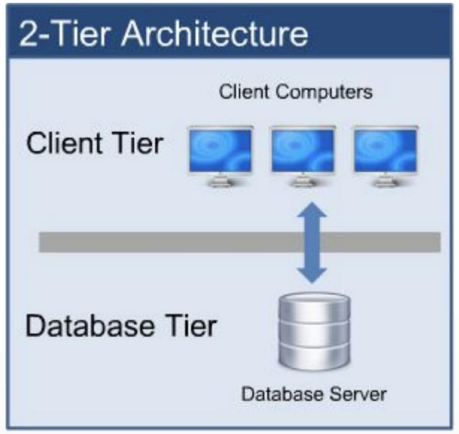


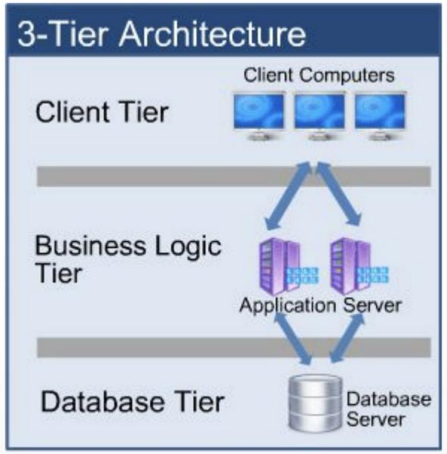


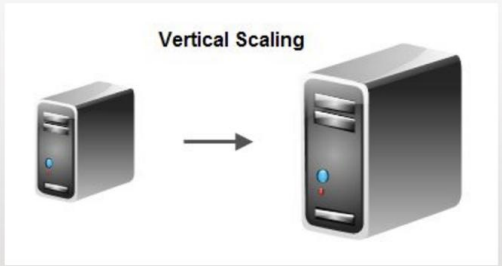
Características

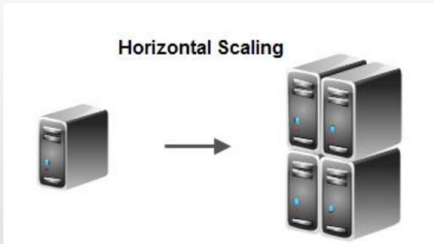
La arquitectura N-Tiers permite una **escalabilidad eficiente** en diferentes niveles:

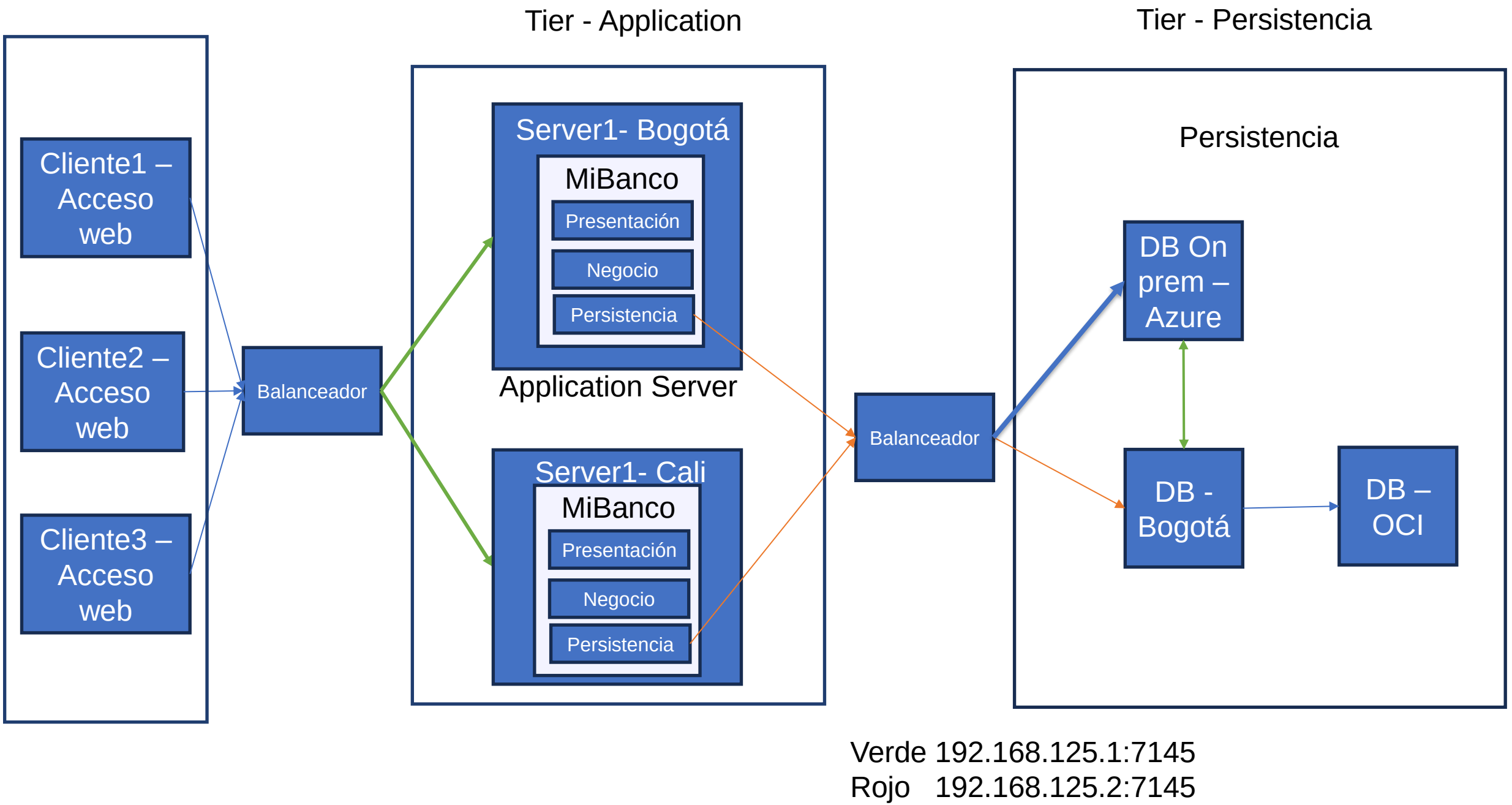
- **Capa de Presentación**: Puedes escalar horizontalmente los servidores web o aplicaciones de front-end para manejar más usuarios.
- **Capa de Lógica de Negocio**: Puedes escalar los servidores de aplicaciones o microservicios para procesar más solicitudes.
- **Capa de Datos**: Puedes escalar la base de datos o utilizar técnicas como la replicación o la partición para manejar más datos y consultas.

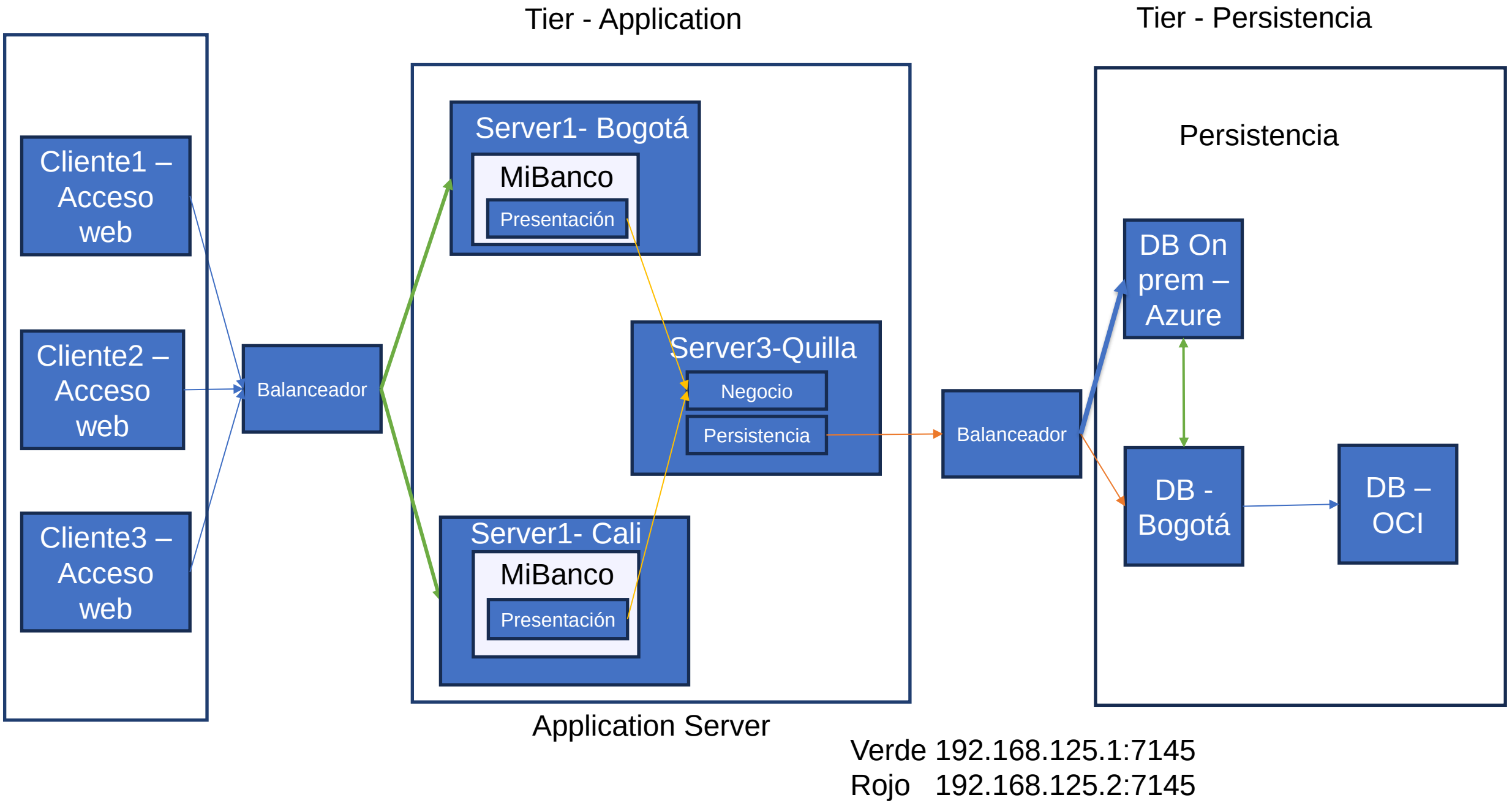


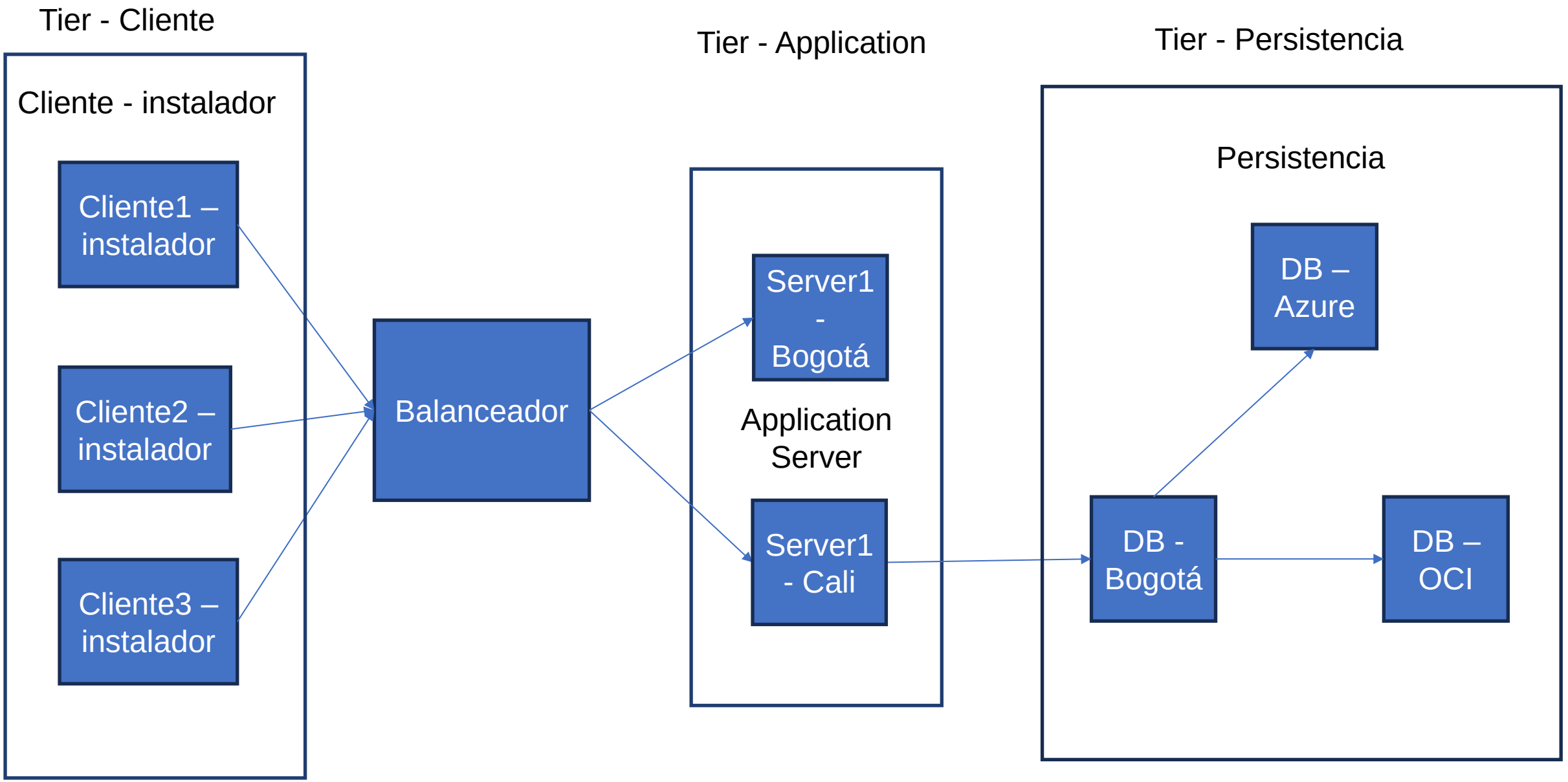







Tier - Application
Tier - Persistencia
Cliente1 – Acceso web
Cliente2 – Acceso web
Cliente3 – Acceso web
Balanceador
Server1- Bogotá
MiBanco
Presentación
Negocio
Persistencia
Application Server
Server1- Cali
MiBanco
Presentación
Negocio
Persistencia
Balanceador
Persistencia
DB On prem – Azure
DB - Bogotá
DB – OCI
Verde 192.168.125.1:7145
Rojo 192.168.125.2:7145


Tier - Application
Tier - Persistencia
Cliente1 – Acceso web
Cliente2 – Acceso web
Cliente3 – Acceso web
Balanceador
Server1- Bogotá
MiBanco
Presentación
Server3-Quilla
Negocio
Persistencia
Server1- Cali
MiBanco
Presentación
Application Server
Balanceador
Persistencia
DB On prem – Azure
DB - Bogotá
DB – OCI
Verde 192.168.125.1:7145
Rojo 192.168.125.2:7145


Tier - Cliente
Cliente - instalador
Cliente1 – instalador
Cliente2 – instalador
Cliente3 – instalador
Balanceador
Tier - Application
Server1 - Bogotá
Application Server
Server1 - Cali
Tier - Persistencia
Persistencia
DB – Azure
DB - Bogotá
DB – OCI



Facultad de Ingeniería

Universidad de La Sabana

Ventajas y Desventajas

Factor	Ventajas	Desventajas
Facilidad de Despliegue	Permite una instalación y actualización sencilla y rápida.	Puede ser complejo si las capas no están bien definidas.
Escalabilidad	Facilita la expansión para soportar más usuarios o características.	Puede ser limitada por la dependencia entre capas.
Facilidad de Implementación	Simplifica el desarrollo al dividir en módulos manejables.	La comunicación entre capas puede complicar la implementación.
Mantenibilidad	Las actualizaciones y correcciones son más fáciles con módulos separados.	La interdependencia puede complicar el mantenimiento.
Portabilidad	Permite mover o reutilizar capas en diferentes sistemas o plataformas.	La compatibilidad entre capas puede ser un desafío.
Rendimiento	Optimización específica para cada capa mejora el rendimiento general.	La comunicación entre capas puede afectar el rendimiento.
Seguridad	Permite enfocar medidas de seguridad específicas por capa.	Las vulnerabilidades pueden surgir en las interfaces entre capas.
Separación de Responsabilidades	Clarifica roles y responsabilidades, facilitando la gestión del proyecto.	Puede llevar a una sobrecarga administrativa si no se maneja bien.
Testeabilidad	Facilita las pruebas al aislar las capas para pruebas individuales.	Las interacciones complejas entre capas pueden complicar las pruebas.
Tolerancia a Fallos	Permite diseñar estrategias de recuperación ante fallos en capas.	La propagación de fallos entre capas puede ser un desafío.


Microkernel

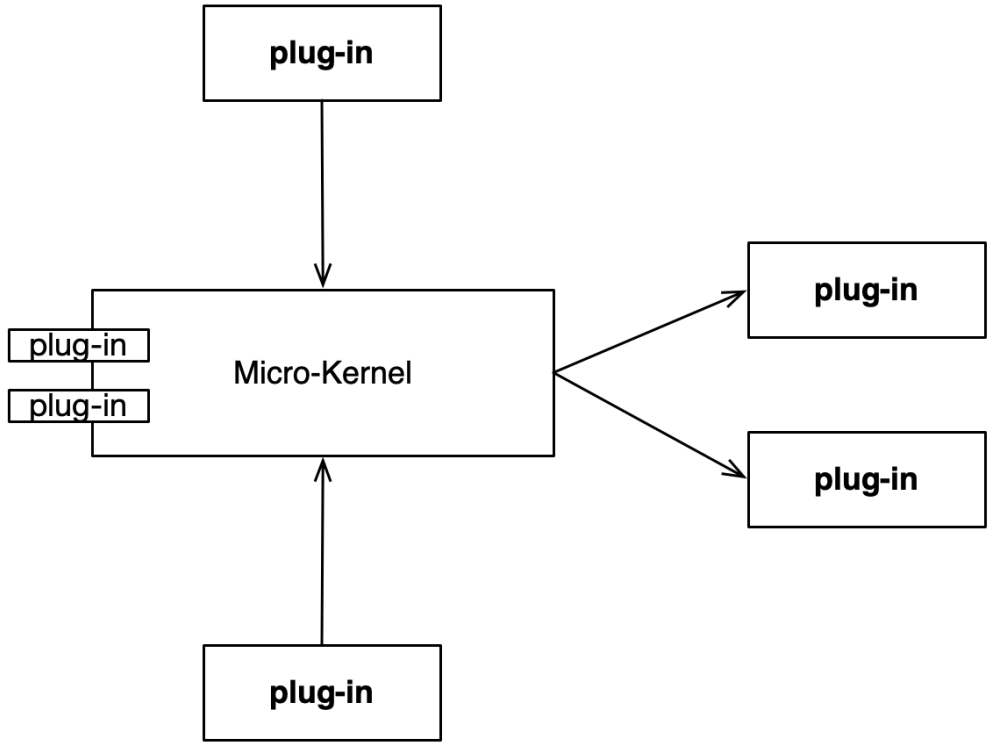


Microkernel

La **arquitectura de Microkernel**, también conocida como arquitectura de Plug-in, permite crear aplicaciones extensibles.

Se basa en la idea de dividir las aplicaciones en dos tipos de componentes: el **sistema Core** (o sistema central) y los **plugins** (o módulos).

El **sistema Core** contiene los elementos mínimos para que la aplicación funcione, mientras que los **plugins** extienden su funcionalidad.



Estructura

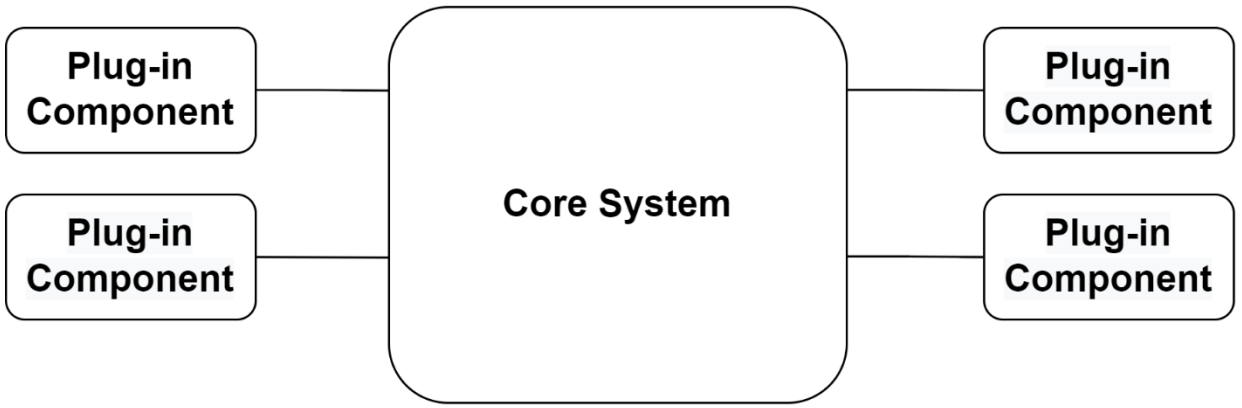
El sistema Core es esencial y no se modifica fácilmente, mientras que los plugins se agregan para ampliar o personalizar la aplicación.

La arquitectura sigue el principio “**Open-Closed**”: abierta para extender la funcionalidad, pero cerrada para modificar su esencia.





Características



Extensibilidad: Permite agregar funcionalidad mediante pequeños plugins.

Separación de preocupaciones: Divide el sistema en Core y plugins.

Dinamismo: Los plugins pueden instalarse y desinstalarse en tiempo de ejecución.



Facultad de Ingeniería

Universidad de La Sabana

Ventajas y Desventajas

Aspecto	Ventajas	Desventajas
Flexibilidad	Permite adaptarse a nuevos requisitos sin alterar el sistema Core.	Desarrollar sistemas con soporte para plugins es complicado.
Mantenibilidad	Cambios en plugins no afectan el sistema base.	Requiere una comunicación clara entre Core y plugins.
Comunidad	Permite a terceros crear plugins y extender la funcionalidad.	
Separación	Divide el sistema en Core y plugins, facilitando la gestión y el mantenimiento.	
Dinamismo	Los plugins pueden instalarse y desinstalarse en tiempo de ejecución.	



Ejemplos

eclipse

IDEs utilizan esta arquitectura.

Programas de ofimática también permiten agregar funcionalidad mediante plugins.

Arquitectura Orientada a Servicios (SOA)

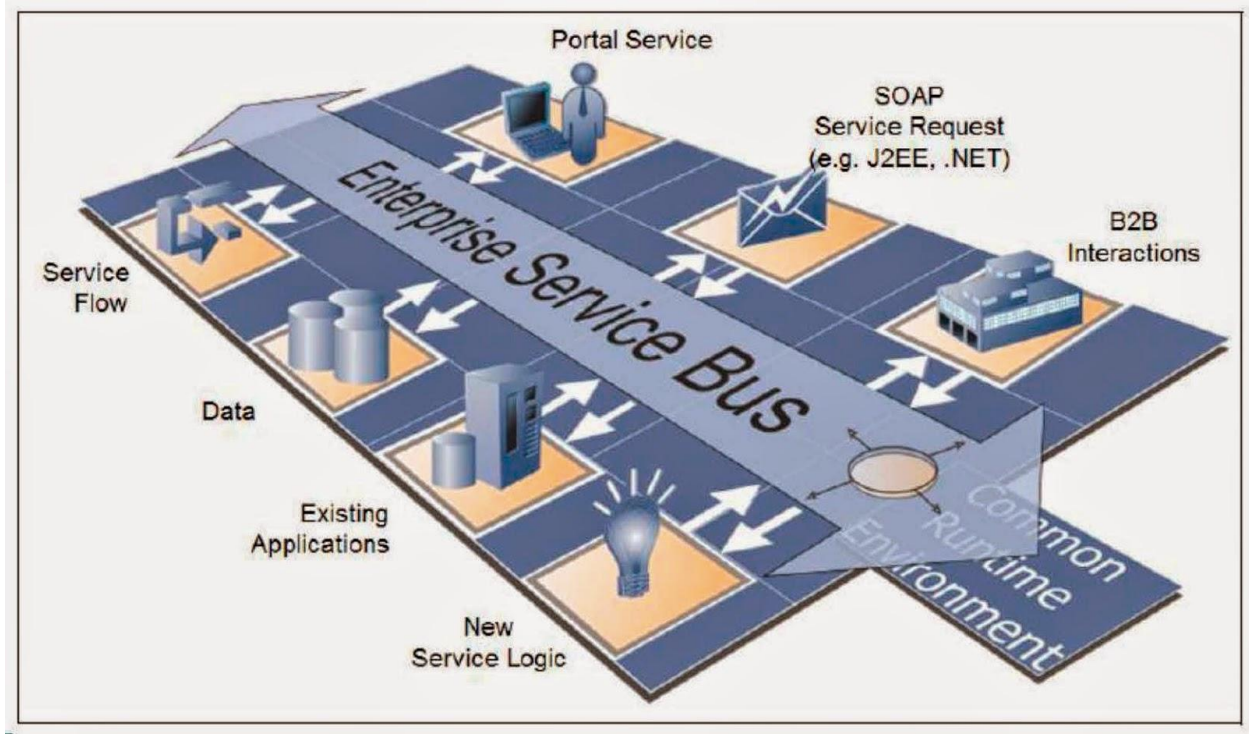




Arquitectura Orientada a Servicios (SOA)

SOA, o **arquitectura orientada a servicios**, define una manera de hacer que los componentes de software sean reutilizables a través de interfaces de servicio.

Cada servicio en una SOA incorpora el código y las integraciones de datos necesarias para ejecutar una función de negocios completa y discreta (por ejemplo, comprobar el crédito de un cliente, calcular un pago de préstamo mensual o procesar una solicitud de hipoteca)



Estructura

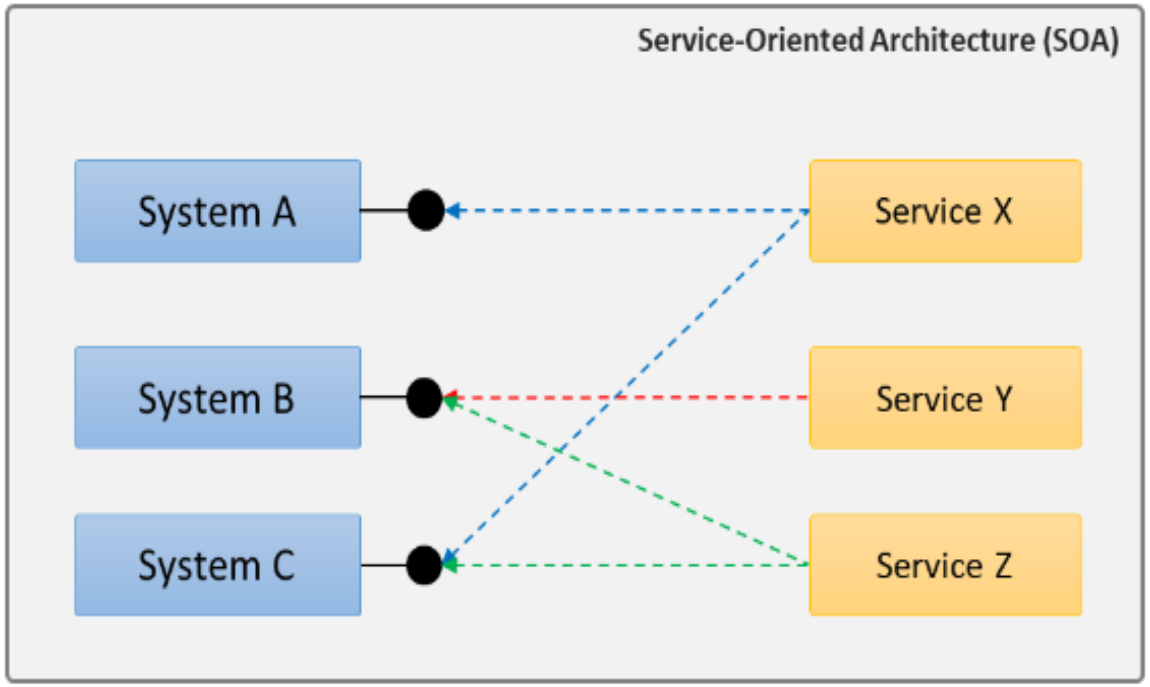


Fig 60 - Consumo de servicios

- SOA se basa en la idea de dividir aplicaciones en servicios independientes.
- Estos servicios se comunican a través de interfaces estándar, lo que permite un acoplamiento suelto entre ellos.
- Las aplicaciones detrás de la interfaz de servicios pueden estar escritas en Java, Microsoft .Net, Cobol u otros lenguajes de programación.





Características



Extensibilidad: Permite agregar nuevos servicios sin afectar otros componentes.

Reusabilidad: Los servicios se pueden reutilizar en diferentes aplicaciones.

Interoperabilidad: Facilita la comunicación entre sistemas heterogéneos.



Enterprise Service Bus (ESB)

Permite que las aplicaciones se comuniquen únicamente con el **Service Bus**, simplificando la integración (**Abstracción de Servicios**).

Control de Comunicación:

Facilita la gestión de la comunicación entre aplicaciones, reduciendo el acoplamiento.

Lógica Adicional:

- Incorpora funciones como comprobación de seguridad y auditoría.
- Mide el número de invocaciones y tiempos de respuesta.
- Detecta y notifica errores en la comunicación.

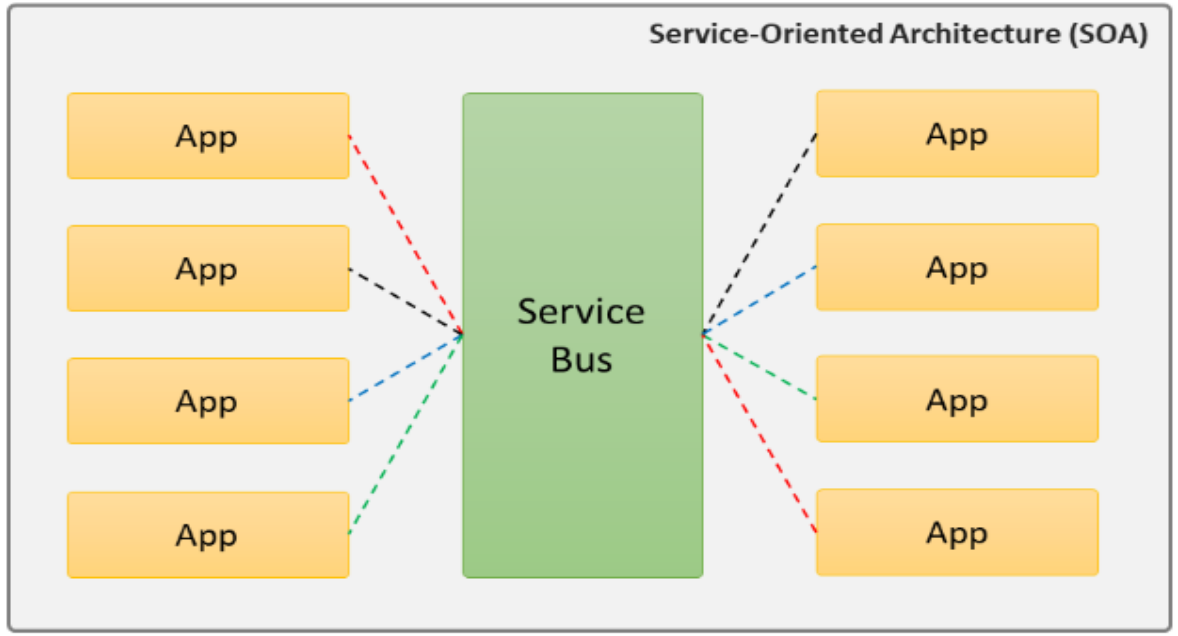


Fig 66 - Arquitectura de un Enterprise Service Bus.



Facultad de Ingeniería

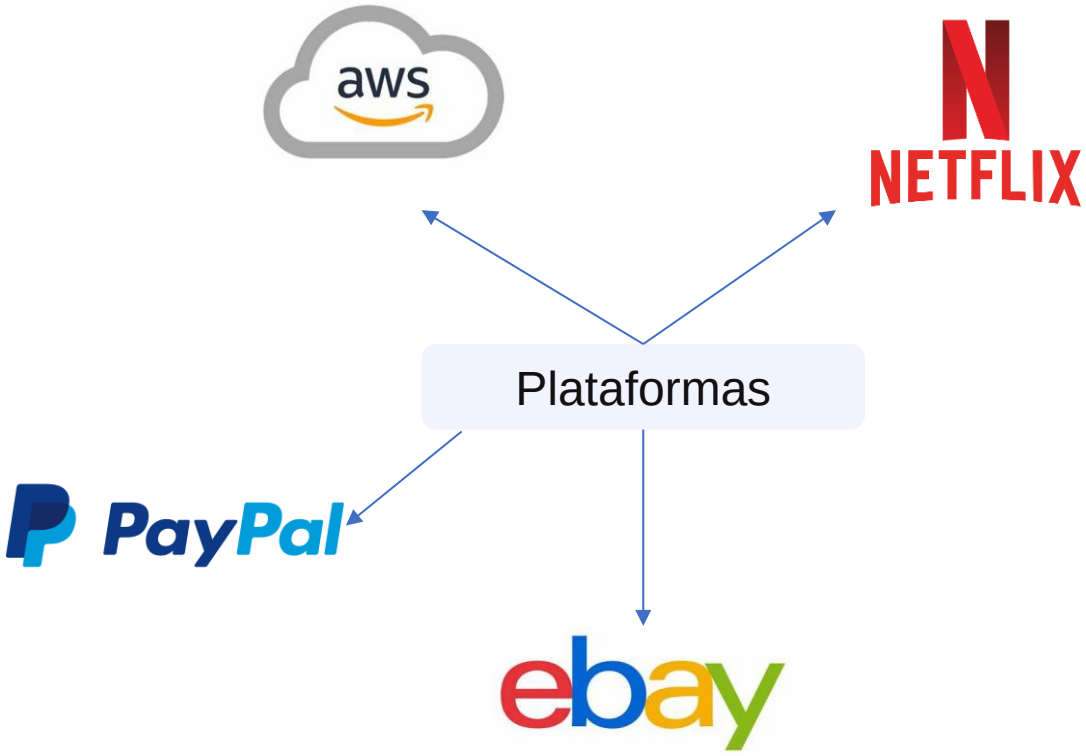
Universidad de La Sabana

Ventajas y Desventajas

Aspecto	Ventajas	Desventajas
Flexibilidad	Permite adaptarse a nuevos requisitos sin alterar el sistema Core.	Diseñar una SOA puede ser complicado.
Reusabilidad	Los servicios se pueden reutilizar en diferentes aplicaciones.	Requiere un enfoque cuidadoso para el ciclo de vida de los servicios.
Interoperabilidad	Facilita la comunicación entre sistemas heterogéneos.	
Reducción del plazo de comercialización	Permite ensamblar aplicaciones rápidamente.	
Mejora del mantenimiento	Cambios en un servicio no afectan otros.	



Ejemplos



Aplicaciones Empresariales



¿Cuándo usar/No usar el estilo SOA?

Usar SOA cuando necesitas:

- Integrar sistemas heterogéneos.
- Reutilizar funcionalidades en diferentes aplicaciones.
- Adaptarte rápidamente a cambios en los requisitos.

No usar SOA cuando:

- Tu aplicación es simple y no requiere interoperabilidad.
- La complejidad inicial supera los beneficios esperados.



Microservicios



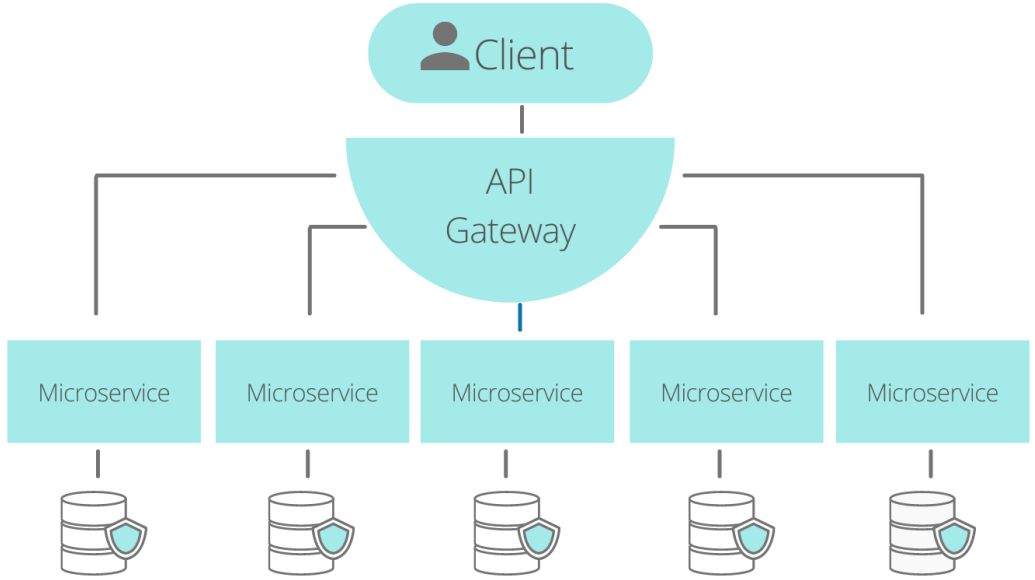


Microservicios

La **arquitectura de microservicios** es un método de desarrollo de aplicaciones software que funciona como un conjunto de **pequeños servicios** que se ejecutan de manera **independiente y autónoma**, proporcionando una funcionalidad de negocio completa.

Cada microservicio es un código que puede estar en un lenguaje de programación diferente y desempeña una función específica.

Los microservicios se comunican entre sí a través de APIs y cuentan con sistemas de almacenamiento propios para evitar sobrecarga y caídas.



Estructura

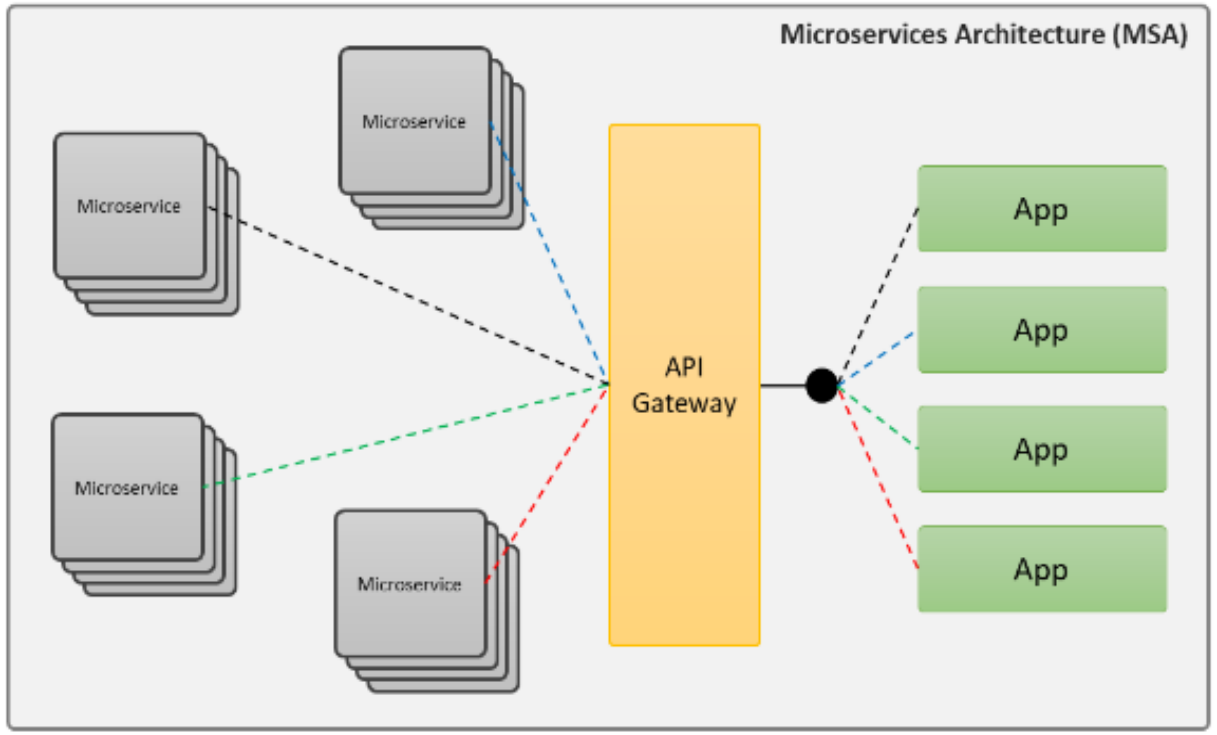


Fig 68 - Arquitectura de Microservicios

División de Funcionalidades:

Separa la funcionalidad de un monolito en componentes pequeños, autosuficientes e independientes, como "mini monolitos".

Componentes Independientes:

Cada microservicio opera de manera autónoma, permitiendo una mayor flexibilidad y escalabilidad.

API Gateway:

- Actúa como puerta de entrada a los microservicios.
- Controla el acceso y las funcionalidades expuestas.



Estructura

Colaboración entre Microservicios:

A veces, un microservicio no puede completar una tarea por sí solo y necesita comunicarse con otros microservicios.

Patrones de Coordinación:

- **Orquestación**: Un coordinador central gestiona y controla la interacción entre microservicios.
- **Coreografía**: Cada microservicio responde a eventos y se coordina con otros sin un control centralizado.

Principios ACID:

- **Atomicidad**: Las transacciones son todo o nada.
- **Consistencia**: El sistema transita de un estado válido a otro.
- **Aislamiento**: Las transacciones se ejecutan de manera independiente.
- **Durabilidad**: Los cambios son permanentes una vez confirmados.

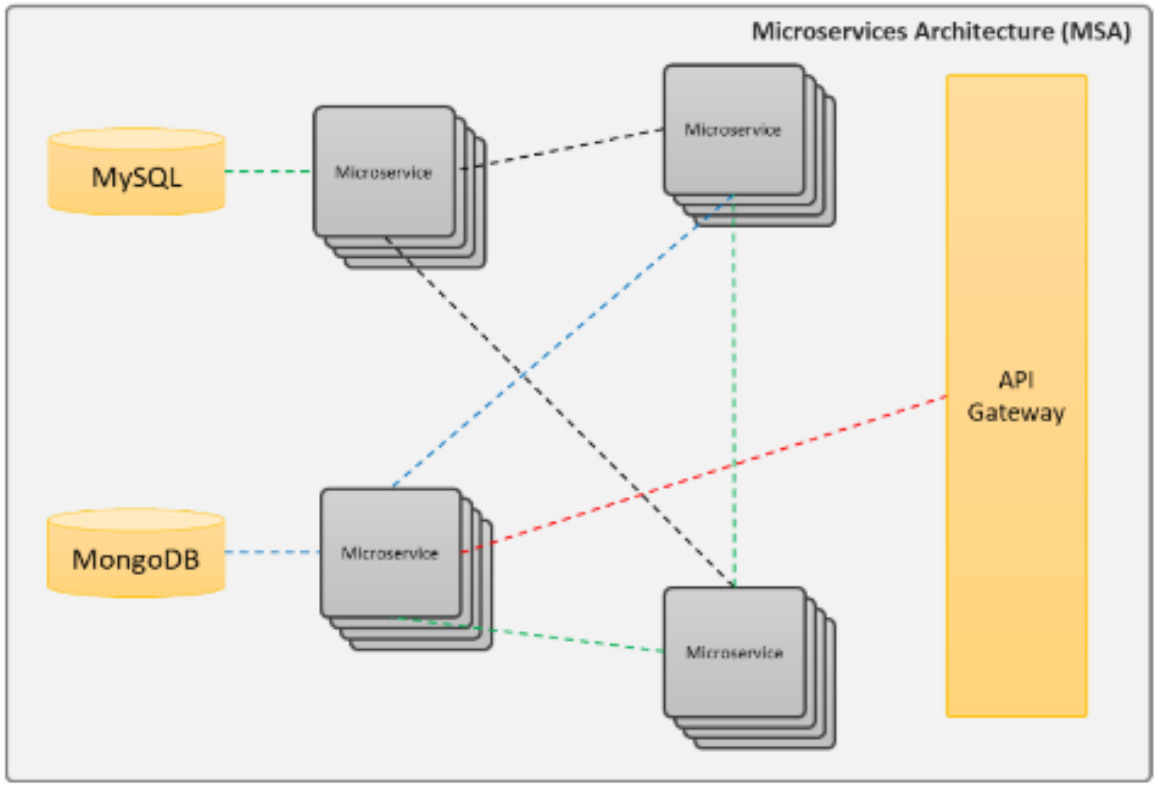


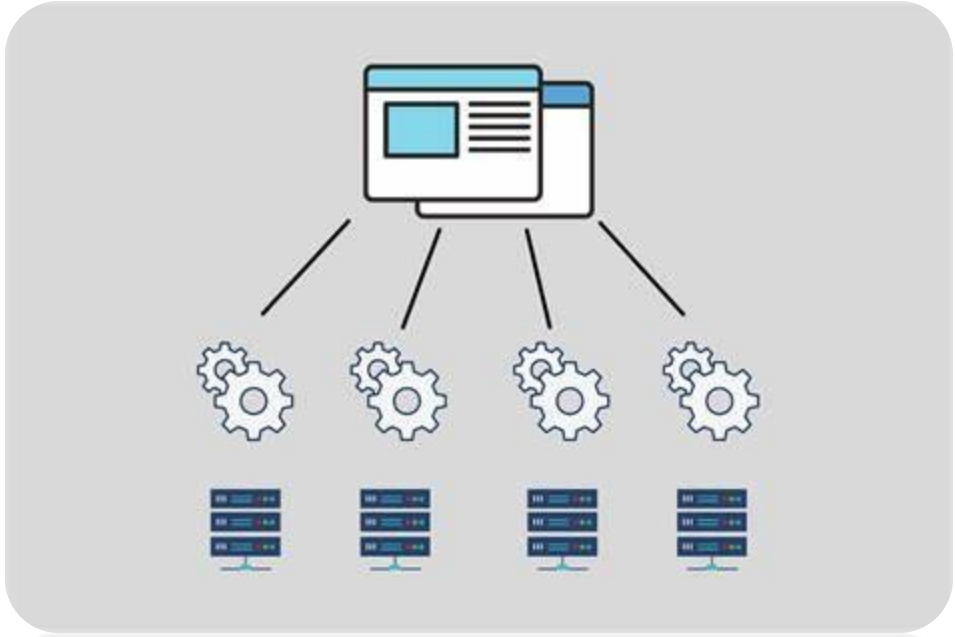
Fig 69 - Comunicación entre Microservicios





Características

- Alta Cohesión
- Autosuficiencia
- Despliegue Independiente
- Alta Disponibilidad
- Arquitectura Compuesta
- Comunicación Estándar



- Diversidad Tecnológica



Facultad de Ingeniería

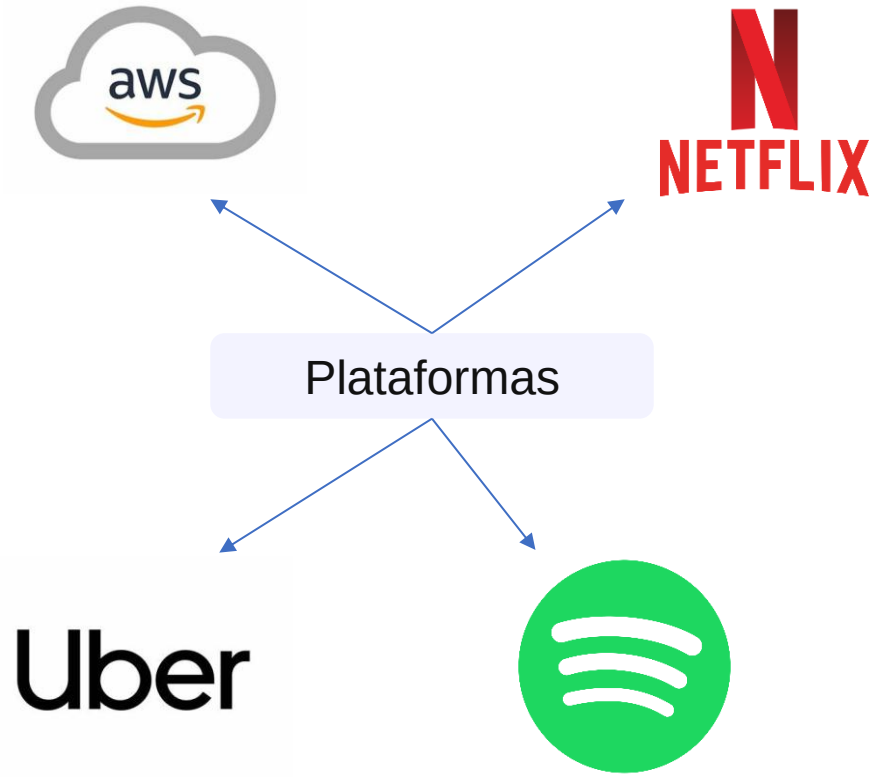
Universidad de La Sabana

Ventajas y Desventajas

Aspecto	Ventajas	Desventajas
Modularidad	Permite desarrollar e implementar servicios de forma independiente.	Complejidad en la comunicación entre microservicios.
Escalabilidad	Escalado específico de componentes sin afectar todo el sistema.	Mayor consumo de memoria debido a la gestión individual de servicios.
Versatilidad	Facilita actualizaciones plug-and-play y cambios rápidos.	Complejidad en la gestión de la configuración y monitoreo.
Depuración	Simplifica la identificación y corrección de errores.	Mayor complejidad en el desarrollo y pruebas.
Adaptabilidad	Permite una entrega rápida y frecuente de aplicaciones complejas.	Despliegue inicial más complejo.



Ejemplos



Aplicaciones de comercio electrónico, redes sociales y servicios en la nube también adoptan esta arquitectura.

¿Cuándo usar/No usar el estilo Microservicios?

Usar microservicios cuando:

- Necesitas escalabilidad y flexibilidad.
- Quieres diversidad tecnológica.

No usar microservicios cuando:

- El proyecto es pequeño o no requiere escalabilidad.
- La complejidad inicial supera los beneficios esperados.





Arquitectura Dirigida por Eventos (EDA)

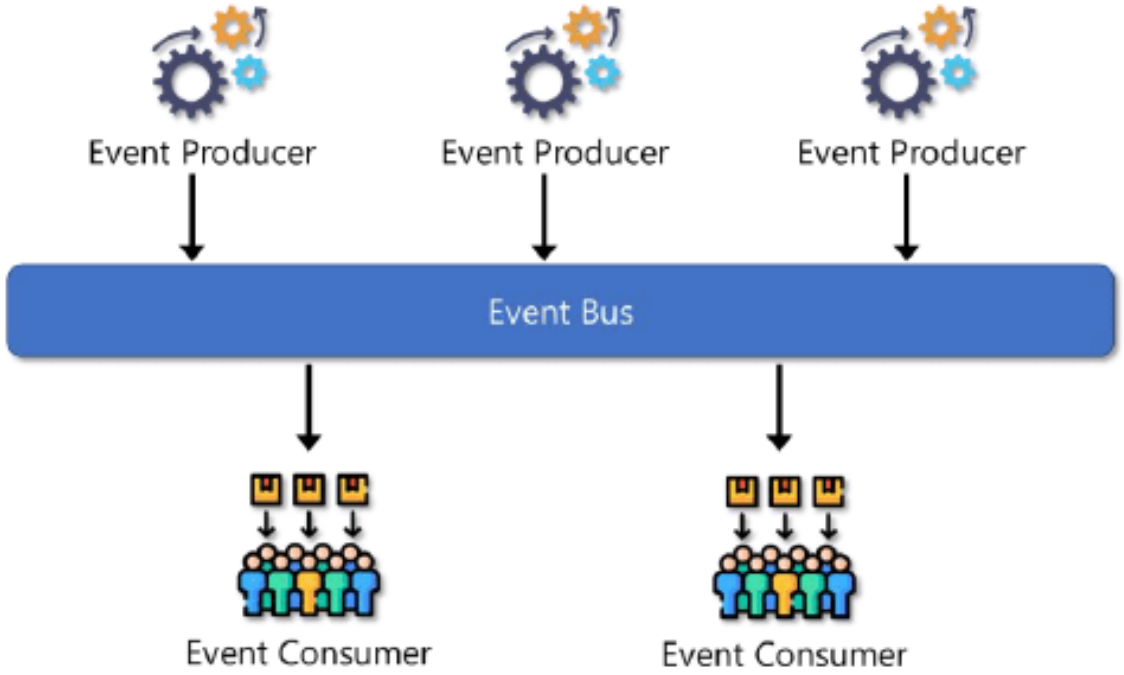


Arquitectura Dirigida por Eventos (EDA)

La **arquitectura dirigida por eventos (EDA)** es un modelo y una arquitectura de software que se utiliza para diseñar aplicaciones. Es una arquitectura asíncrona y distribuida, pensada para crear aplicaciones altamente escalables.

Su objetivo es registrar, comunicar y procesar **eventos** entre servicios desacoplados.

Los eventos pueden ser **cambios significativos** en el estado del hardware, software o acciones específicas en el sistema.



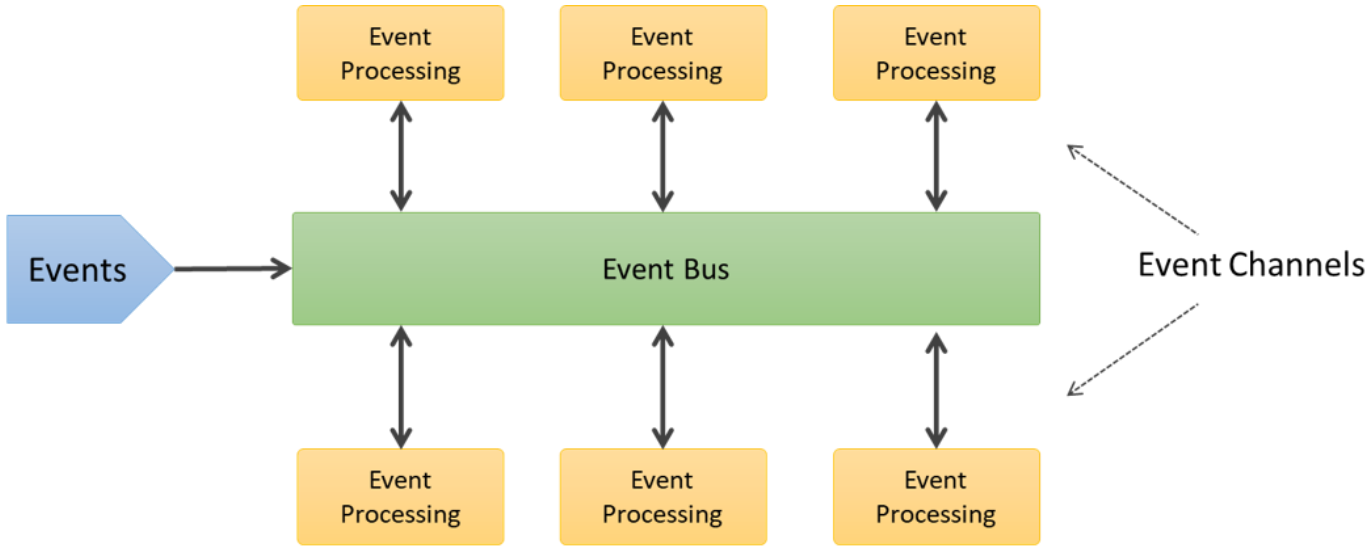


Bus de Eventos

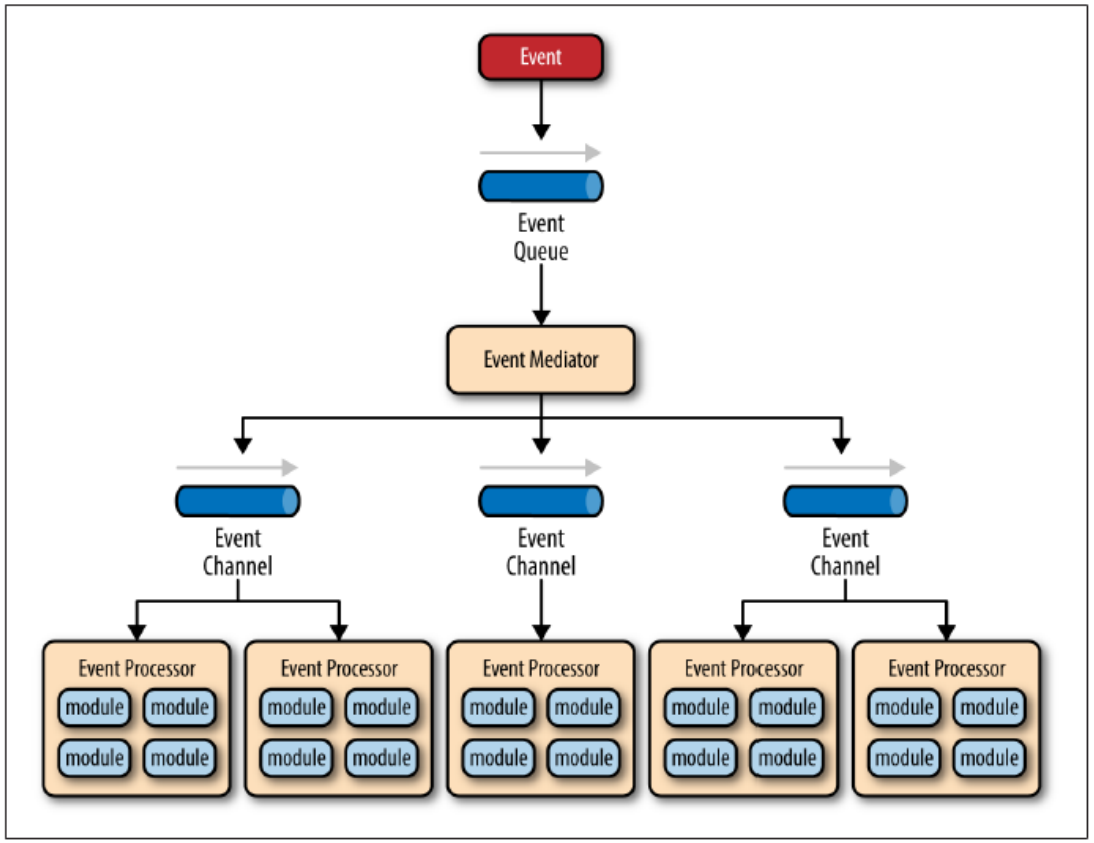
Un bus de eventos es un mecanismo que permite la **transmisión de eventos** entre distintos componentes de una arquitectura de software de manera **desacoplada**.

Canal de eventos:

Actúa como una cola de solicitudes. Permite que uno o varios componentes escuchen y respondan a estos eventos.



Funcionamiento



Event Processing:

Son los componentes que escuchan y responden a los eventos.

Acciones Específicas:

Realizan tareas concretas en el sistema, como crear, actualizar o eliminar registros.

Distribución y Especialización:

Están distribuidos y son altamente especializados en sus funciones.
Operan de manera independiente sin conocimiento de otros procesadores de eventos.



Estructura - Broker

Características:

- Utilizada para procesar eventos simples que no requieren una orquestación compleja.
- Los Event Processors toman mensajes directamente del Event Channel.

Funcionalidad:

Pueden publicar nuevos eventos para que otros Event Processors continúen con el trabajo.

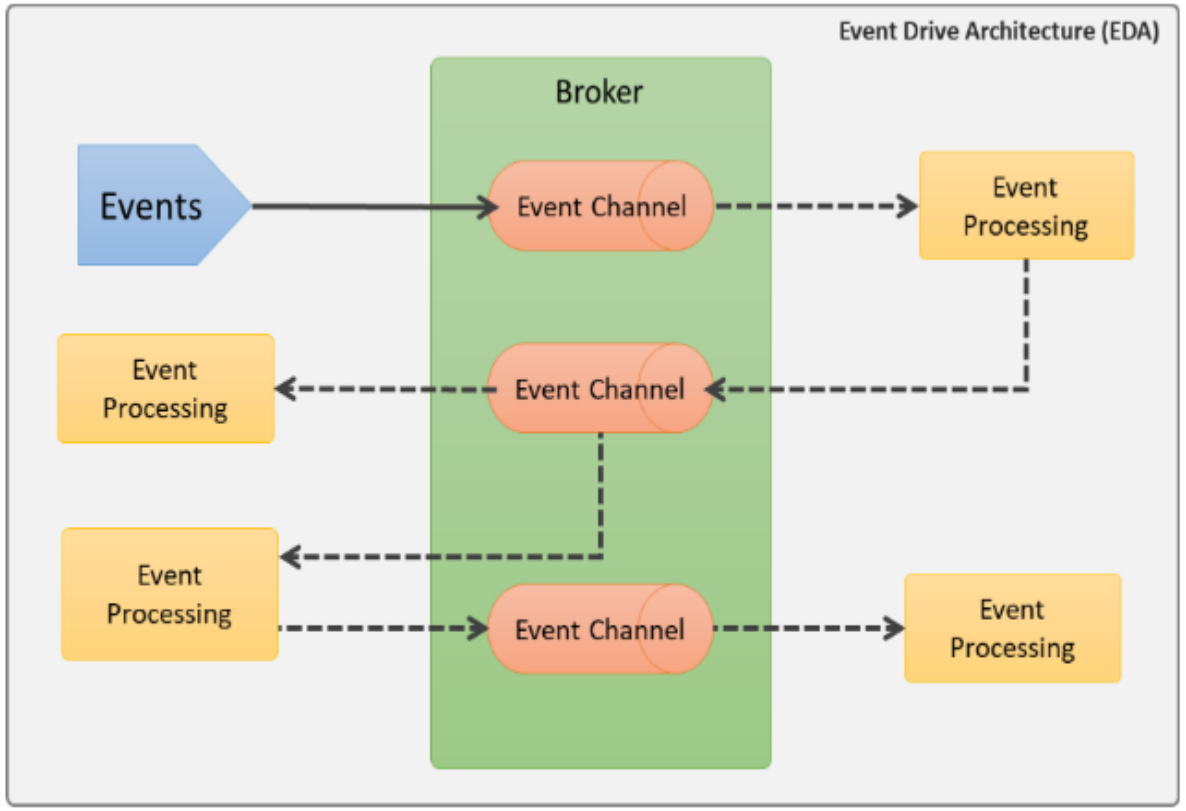


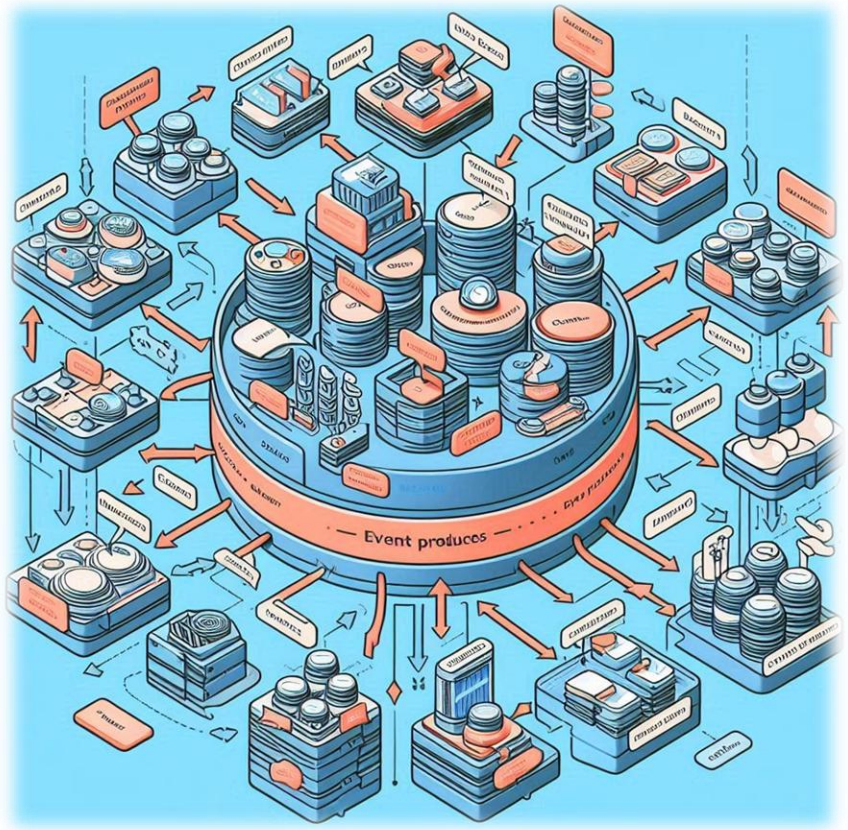
Fig 75 - Topología con Broker.





Características

- Componentes Distribuidos
- Mensajería Asíncrona
- Componente Intermediario



- Compatibilidad con SOA y Microservicios
- Operaciones de Negocio Desglosadas
- Desacoplamiento Total



Facultad de Ingeniería

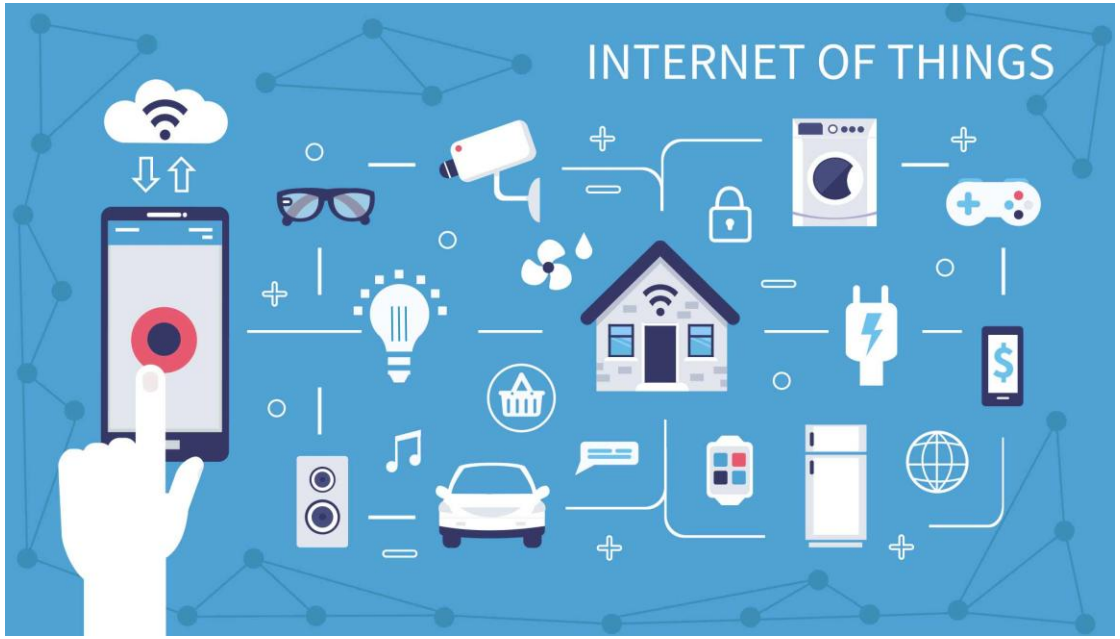
Universidad de La Sabana

Ventajas y Desventajas

Aspecto	Ventajas	Desventajas
Bajo Acoplamiento	Reduce la dependencia entre componentes.	Complejidad en la gestión de eventos y sus relaciones.
Adaptabilidad	Permite reaccionar ágilmente a eventos cambiantes.	Mayor complejidad en la lógica de adaptación a eventos.
Desplegabilidad	Facilita la implementación y actualización de componentes independientes.	Requiere una infraestructura adecuada para la distribución de eventos.
Escalabilidad	Maneja grandes volúmenes de eventos y crecimiento sin afectar todo el sistema.	Mayor consumo de recursos debido a la gestión de eventos.
Facilidad de desarrollo	Simplifica la identificación y corrección de errores.	Mayor complejidad en la implementación de la lógica de eventos.
Testeabilidad	Facilita las pruebas unitarias y de integración.	Requiere pruebas específicas para la gestión de eventos.



Ejemplos



...Tiempo Real



¿Cuándo usar/No usar el estilo EDA?

Usar EDA cuando:

- Se requiere escalabilidad y flexibilidad.
- Los eventos ocurren con frecuencia y de manera no determinista.
- Reactividad y Tiempo Real

No usar EDA cuando:

- Operaciones Secuenciales y Síncronas
- Sistemas que Requieren Transacciones ACID



Representational State Transfer (REST)





Arquitectura Representational State

REST es un conjunto de **restricciones** que define un estilo arquitectónico comúnmente utilizado para crear aplicaciones distribuidas.

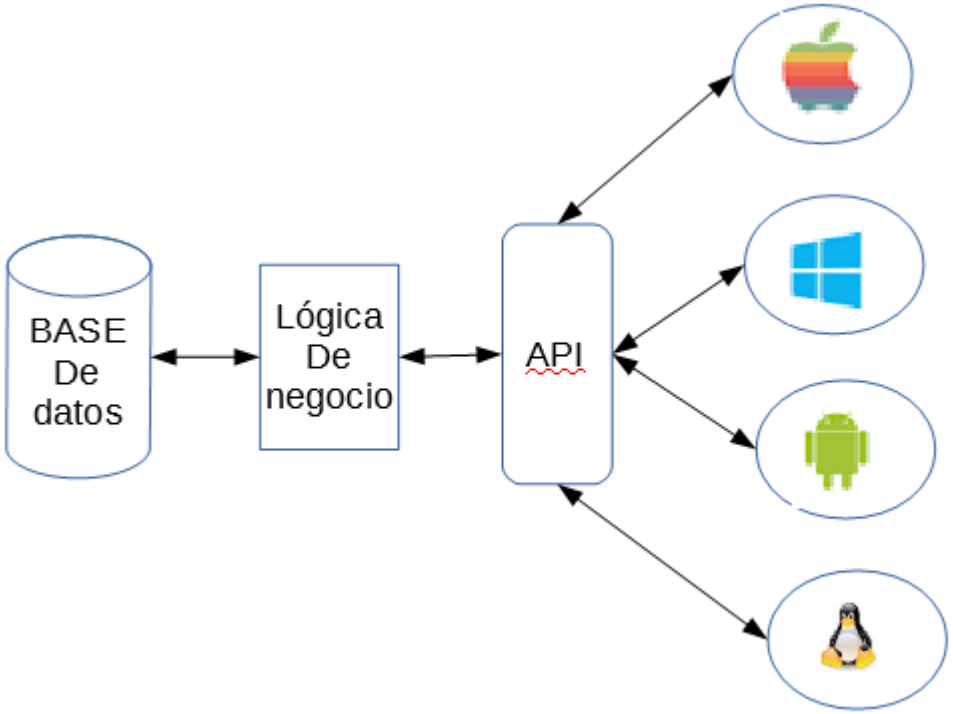
Simplicidad y Escalabilidad

Se basa en la simplicidad, la escalabilidad y la uniformidad en la comunicación entre sistemas distribuidos a través de la web.

Desacoplamiento de Implementación

Ignora los detalles de implementación de los componentes, permitiendo una mayor flexibilidad y evolución del sistema.

Funcionamiento



Fundamento Ampliamente Utilizado:

Es la base sobre la que están construidas la mayoría de las APIs de las principales empresas a nivel mundial.

Uso de REST:

- Se utiliza cuando necesitamos crear un API que permita la interacción con otras plataformas, tecnologías, dispositivos, aplicaciones o proveedores externos.
- Proporciona flexibilidad para trabajar con múltiples representaciones de los datos, independientemente de la tecnología utilizada.



Estructura REST - datos

Concepto:

Los datos se transmiten en bruto desde el servidor al cliente, en lugar de ser transmitidos ya procesados.

Funcionamiento:

El servidor envía los datos sin manipular, permitiendo que el cliente determine cómo procesarlos y utilizarlos.

Beneficios:

- Flexibilidad para adaptar los datos a las necesidades específicas del cliente.
- Reduce la carga de procesamiento en el servidor al delegar parte del trabajo al cliente.

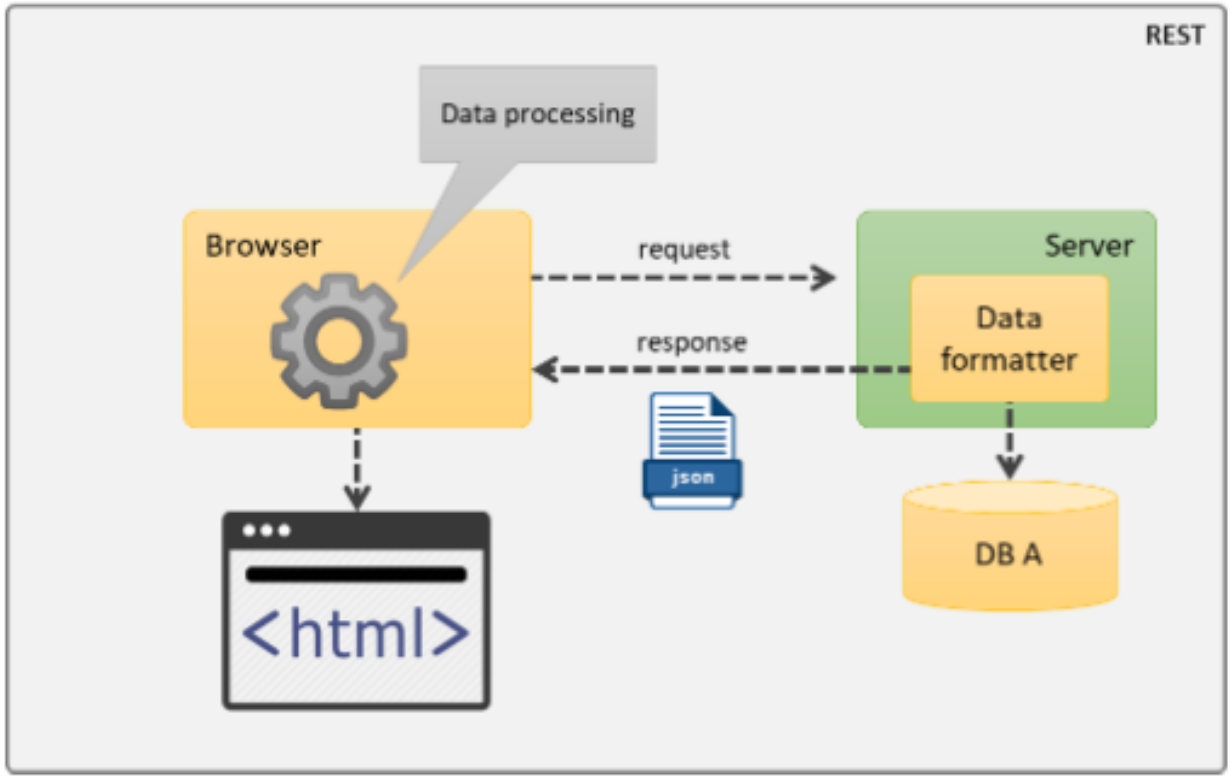


Fig 77 - Procesamiento del lado del cliente



Estructura REST - Recursos

Definición de Recursos:

Un recurso puede ser cualquier entidad (datos, documento, imagen, servicio, colección de recursos, etc.) que pueda ser recuperada mediante una referencia de hipertexto (URL).

Dirección Única:

Cada recurso tiene una dirección única (URL) que lo identifica, asegurando que ningún otro recurso comparta la misma referencia.

Representación del Recurso:

Cada recurso tiene una representación, que es una descripción en un formato específico como JSON o XML.

Formatos Legibles:

Estos formatos son legibles tanto por humanos como por máquinas, lo que facilita la interoperabilidad y el procesamiento de datos.







Características

Identificación Única de Recursos



Independencia de Tecnología

Comunicación Stateless



Facultad de Ingeniería

Universidad de La Sabana

Ventajas y Desventajas

Aspecto	Ventajas	Desventajas
Flexibilidad	Permite a los desarrolladores crear APIs personalizadas según necesidades específicas.	Puede ser difícil de comprender y configurar para principiantes.
Escalabilidad	Maneja grandes cantidades de datos sin sobrecargar el servidor.	
Simplicidad	Utiliza estándares web ampliamente conocidos.	
Desacoplamiento	Los sistemas cliente y servidor se comunican de forma independiente.	
Formato de Intercambio	Utiliza JSON u otros formatos para intercambiar información.	



Ejemplos

Plataformas

Aplicaciones web

¿Cuándo usar/No usar el estilo REST?

Usar REST cuando:

- Necesitas una arquitectura simple y escalable.
- Interoperabilidad
- Estado Stateless
- Simplicidad y Flexibilidad

No usar REST cuando:

- La aplicación requiere transacciones complejas o seguridad avanzada.
- Operaciones que Requieren Alta Eficiencia
- Necesidades de Actualización en Tiempo Real





Patrones arquitectonicos

Segregación de responsabilidad de comandos y consultas (CQRS)



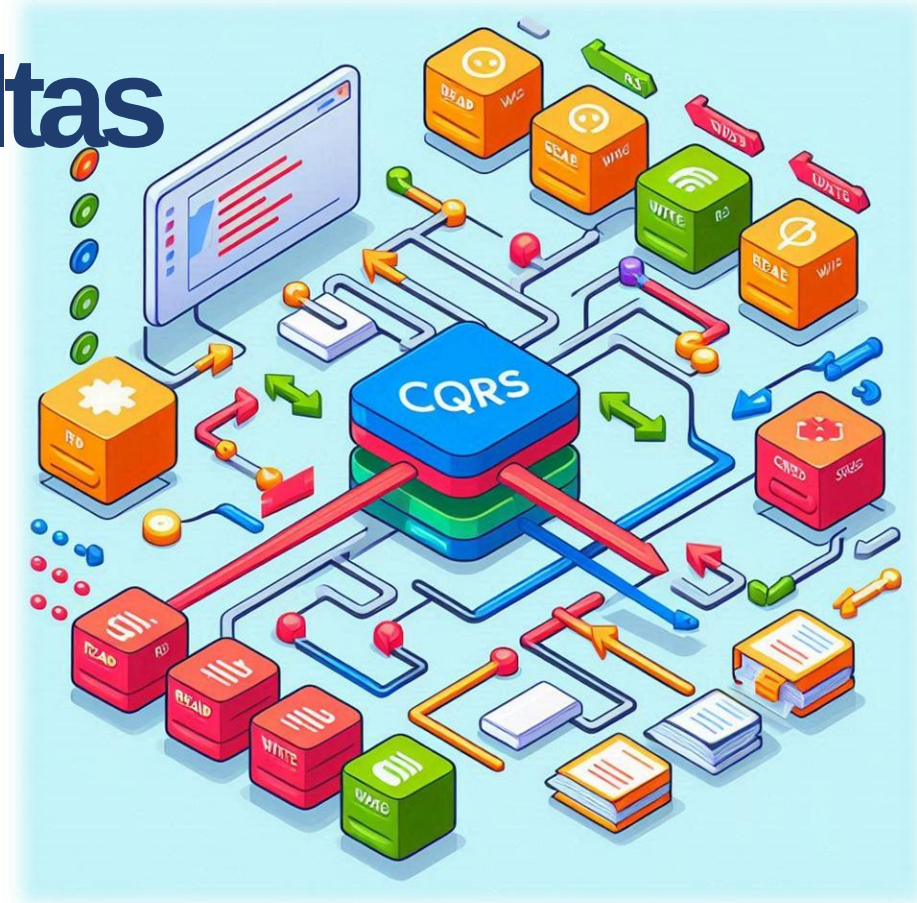


Segregación de responsabilidad de comandos y consultas

Es un **patrón** que separa las operaciones de lectura y actualización en un almacén de datos.

Los comandos se utilizan para actualizar datos, mientras que las consultas solo leen datos.

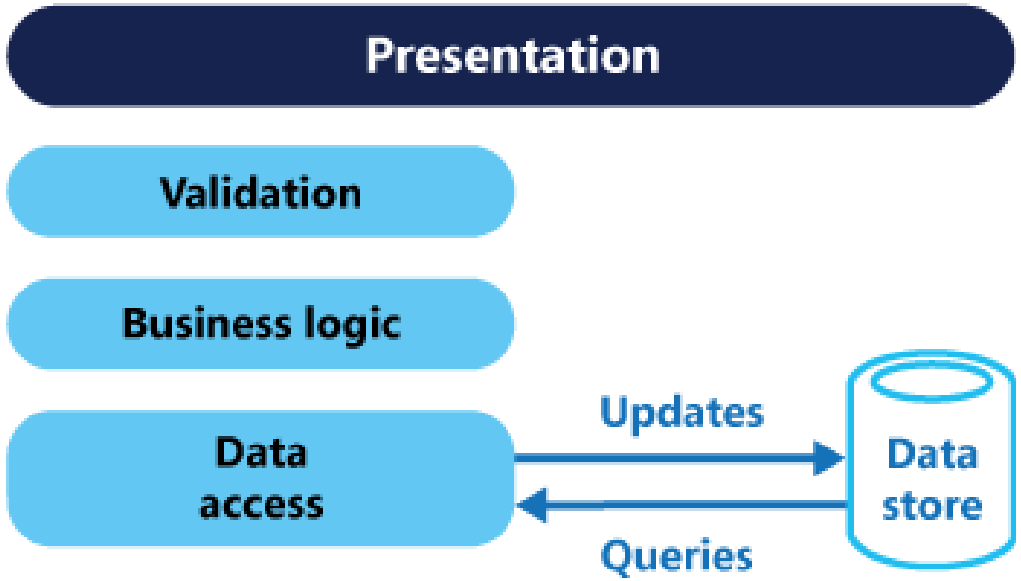
Proporciona una mayor flexibilidad y evita conflictos de fusión a nivel de dominio.





Problema

En las arquitecturas tradicionales, se utiliza el mismo modelo de datos para consultar y actualizar una base de datos. Esto es simple y funciona bien para operaciones CRUD básicas.

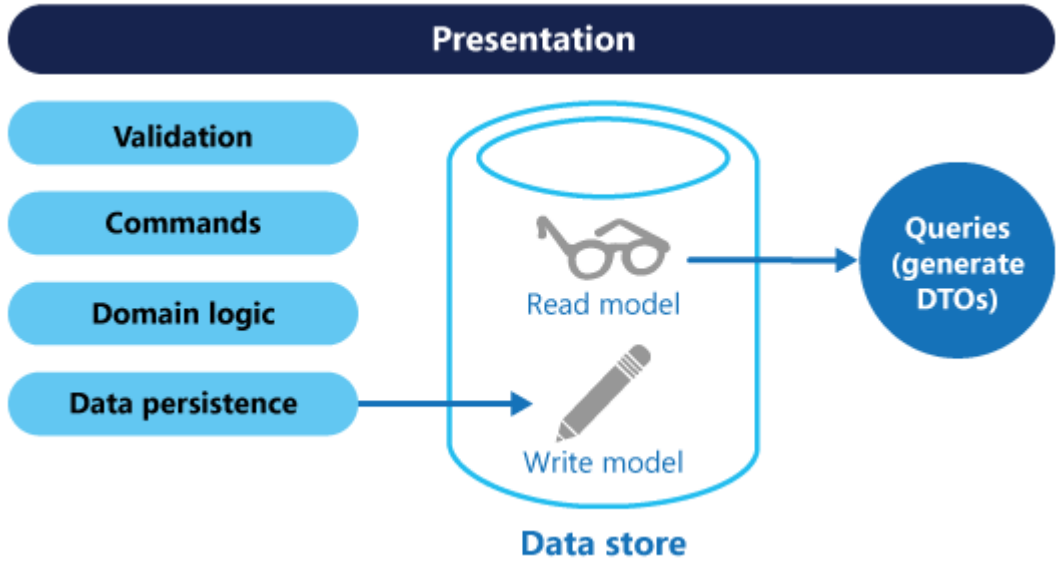


Una arquitectura CRUD tradicional

Solución

Separa las operaciones de lectura y escritura en diferentes modelos. Utiliza **comandos** para actualizar datos y **consultas** para leer datos.

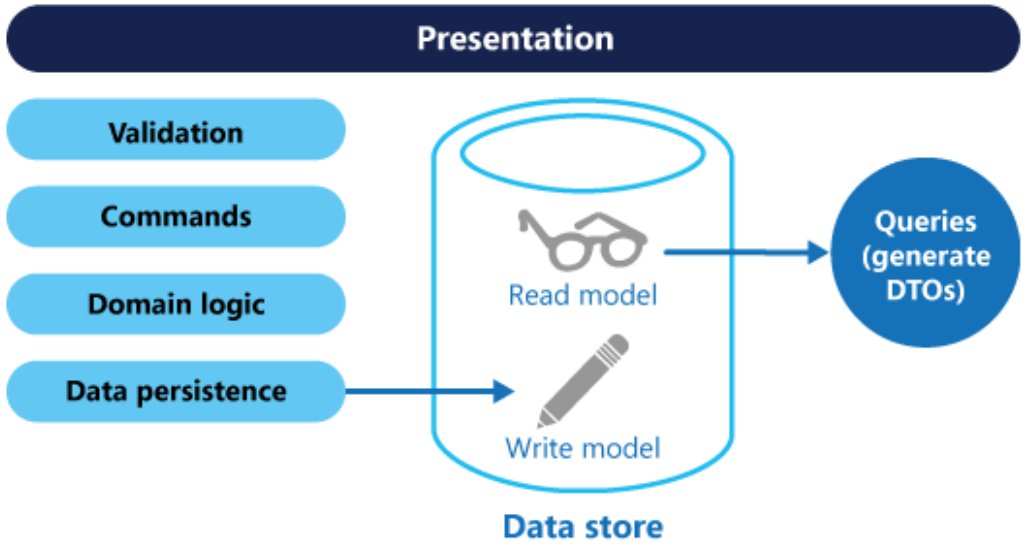
- Los comandos deben estar basados en tareas, no en datos específicos.
- Cambios en la interacción del usuario pueden ser necesarios.



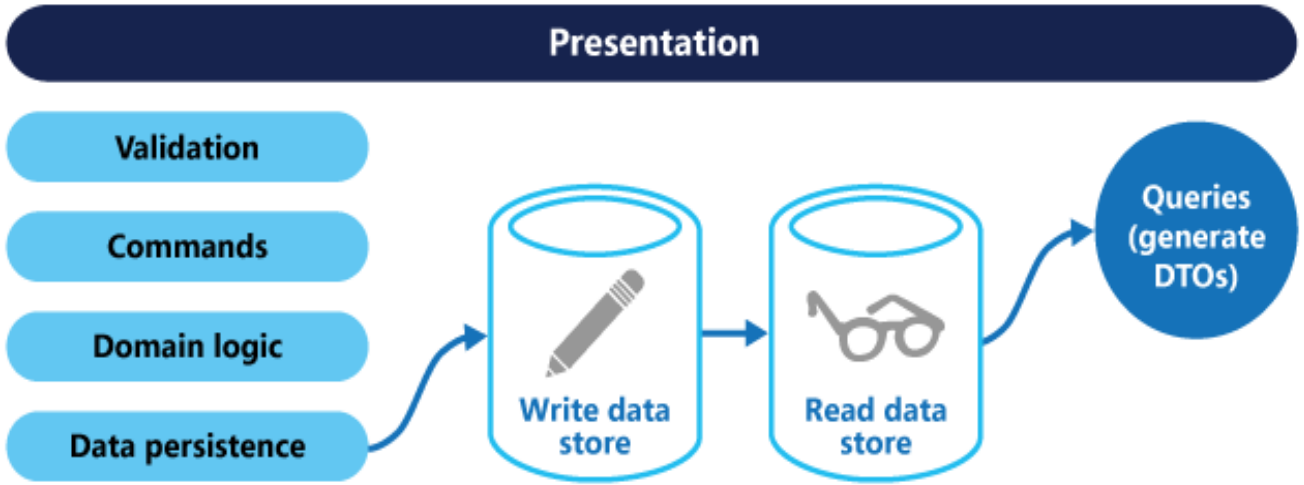


Solución

Los modelos pueden aislarse, aunque no es un requisito absoluto, separar los modelos de consulta y actualización simplifica el diseño y la implementación.



Arquitectura básica de CQRS



Almacenes de lectura y escritura separados





Consideraciones

Réplicas de solo lectura

- El almacén de lectura puede ser una réplica de solo lectura del almacén de escritura.
- Múltiples réplicas de solo lectura pueden mejorar el rendimiento de las consultas, especialmente en escenarios distribuidos.

Separación de almacenes

- Separar los almacenes de lectura y escritura permite ajustar cada uno a la carga adecuada.
- Los almacenes de lectura suelen enfrentar una carga más alta que los almacenes de escritura.

Patrón de Event Sourcing

- El estado de la aplicación se almacena como una secuencia de eventos.
- Los eventos se utilizan para notificar a otros componentes, especialmente al modelo de lectura.
- El modelo de lectura crea una instantánea del estado actual basada en los eventos, lo que mejora la eficiencia de las consultas.

Beneficios de CQRS

Escalabilidad independiente

Esquemas de datos optimizados



Seguridad

Separación de preocupaciones

Consultas más sencillas





Consideraciones de Implementación

Complejidad



Consistencia eventual

¿Cuándo usar CQRS?

Interfaces de usuario basadas en tareas

- Cuando los usuarios siguen un proceso complejo en una serie de pasos o con modelos de dominio complejos.
- El modelo de escritura tiene una pila de procesamiento de comandos y un conjunto de objetos como una unidad para cambios de datos (DDD) y que estén siempre consistentes
- El modelo de lectura devuelve un DTO

Afinar el rendimiento de lecturas y escrituras por separado

- Cuando el rendimiento de las lecturas debe ajustarse por separado del rendimiento de las escrituras, especialmente cuando el número de lecturas es mucho mayor que el de escrituras.



¿Cuándo usar CQRS?

Dominios colaborativos

- Cuando muchos usuarios acceden a los mismos datos en paralelo.

Evolución del sistema y cambios en las reglas de negocio

- Cuando se espera que el sistema evolucione con el tiempo.

Equipos de desarrollo especializados

- Cuando el equipo de desarrollo puede centrarse en algún modelo en específico.

Integración con otros sistemas

- Combinación con el event sourcing



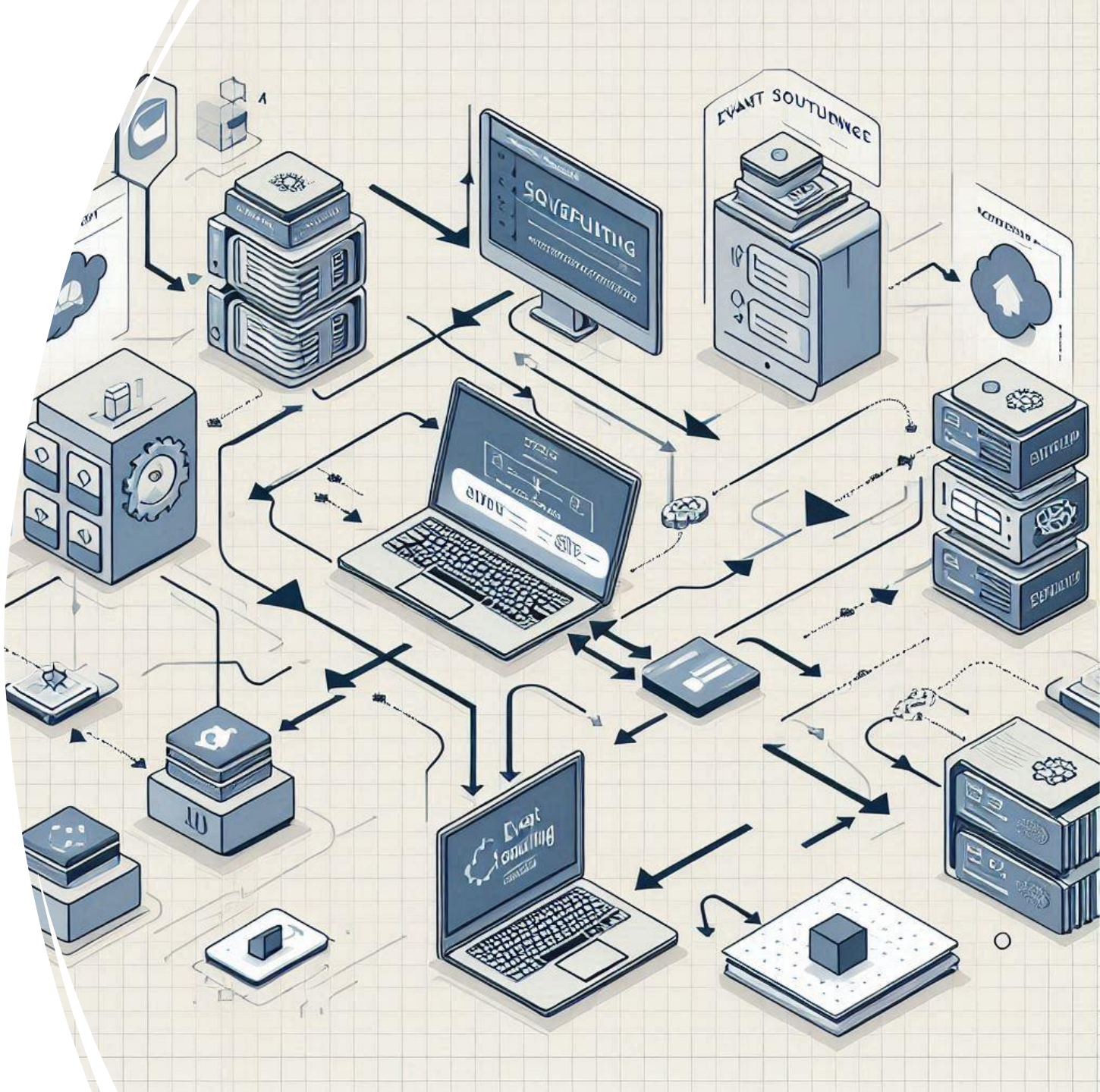
¿Cuándo no se recomienda?

El dominio o las reglas de negocio son simples.

Una interfaz de usuario y operaciones de acceso a datos al estilo CRUD son suficientes.

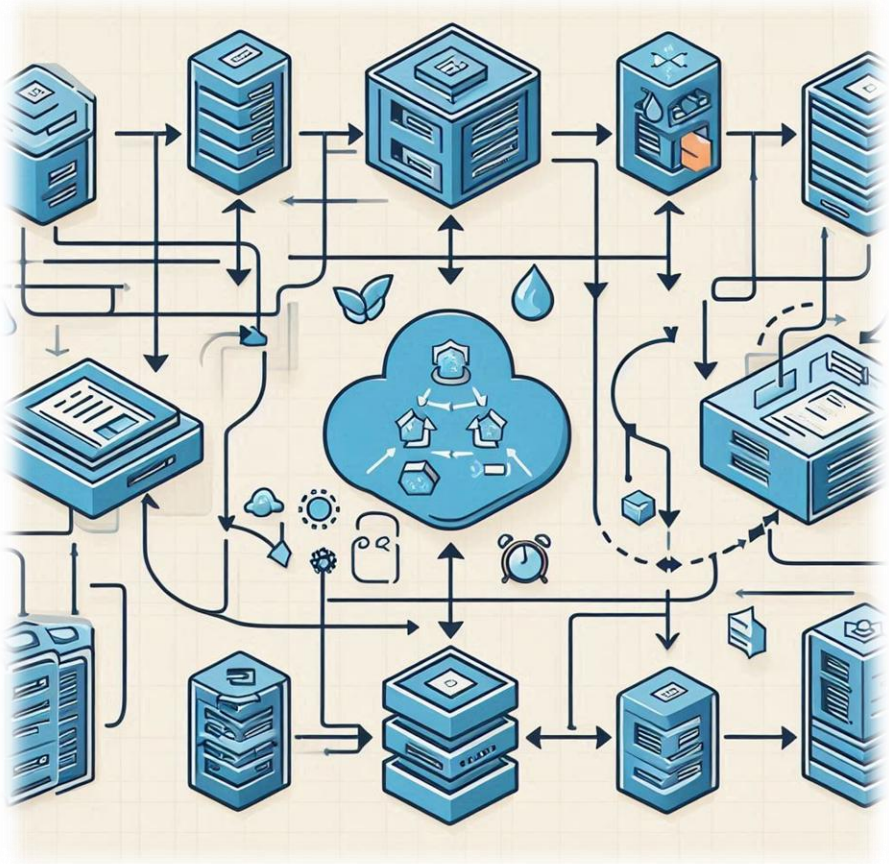


Patrón Event Sourcing





Patrón Event Sourcing



El **patrón Event Sourcing** se utiliza para registrar y controlar las operaciones basadas en una secuencia de eventos en un almacén de solo anexar.

Tiene ventajas como la escalabilidad, la consistencia transaccional y la capacidad de mantener un historial completo de acciones. Sin embargo, también agrega complejidad al diseño del sistema.



Problema



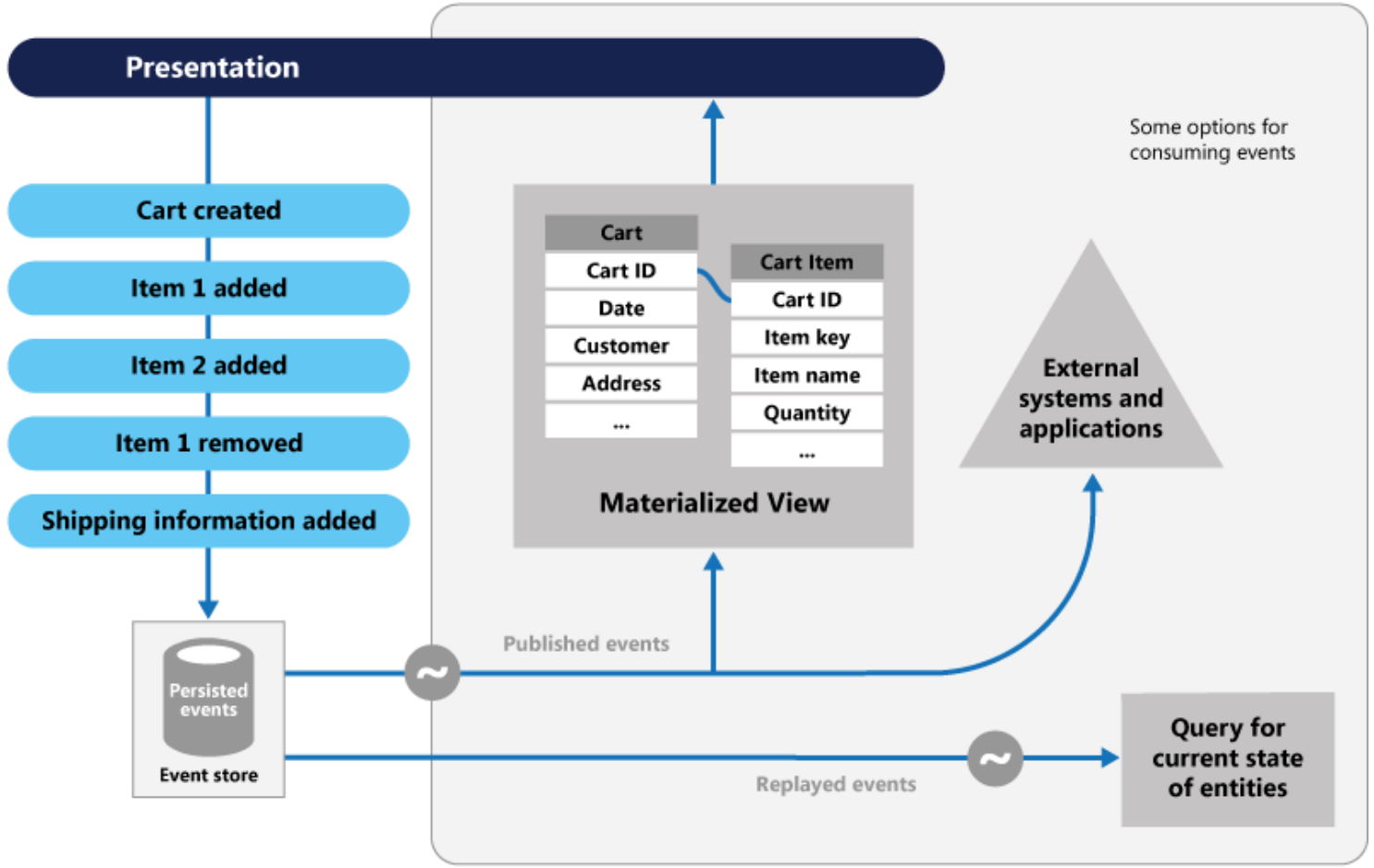
Solución

Funcionamiento

- Los eventos se persisten y el almacén de eventos actúa como sistema de registro.
- Los eventos se publican para que los consumidores los manejen según sea necesario.

Mantener Vistas Materializadas

Integración con Sistemas Externos





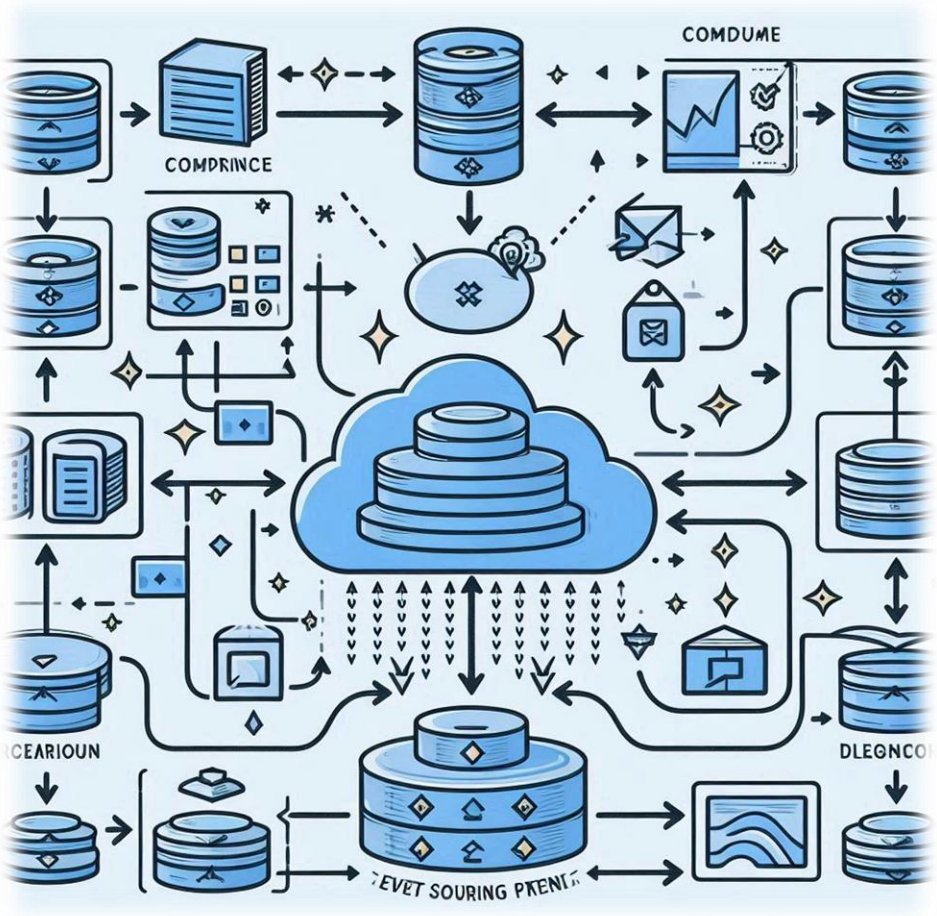
Ventajas del Event Sourcing

Eventos Inmutables

Almacenamiento de solo anexar mejora rendimiento y escalabilidad.

Simplicidad y Gestión

Eventos simplifican la implementación y gestión.



Prevención de Conflictos

Evita conflictos de actualizaciones concurrentes.

Pista de Auditoría

Proporciona un historial detallado de las acciones.





Consideraciones

- Consistencia eventual
- Inmutabilidad del almacén de eventos
- Aplicaciones multi-hilo y consistencia de eventos
- Lectura de eventos
- Gestión de flujos de eventos largos
- Manejo de inconsistencias
- Idempotencia en la publicación de eventos
- Soporte del almacén de eventos
- Evitar bucles infinitos

¿Cuándo usar Event Sourcing?

Captura de intención y propósito

Captura detallada de acciones específicas

Minimización de conflictos

Reducción de conflictos en las actualizaciones de datos

Registro y reproducción de eventos

Historial completo y reproducción de estados



¿Cuándo usar CQRS?

Uso natural de eventos

Integración natural de eventos

Desacoplamiento del proceso de entrada

Desacoplamiento de entrada y procesamiento

Flexibilidad en el formato de modelos materializados

Adaptabilidad del formato de datos

Uso con CQRS

Complemento con CQRS y consistencia eventual



¿Cuándo no se recomienda?

Sistemas sin necesidad de historial o auditoría

Innecesario sin requisitos de historial

Baja incidencia de actualizaciones conflictivas

Poco beneficio en entornos con bajas actualizaciones conflictivas.

Dominios pequeños o simples

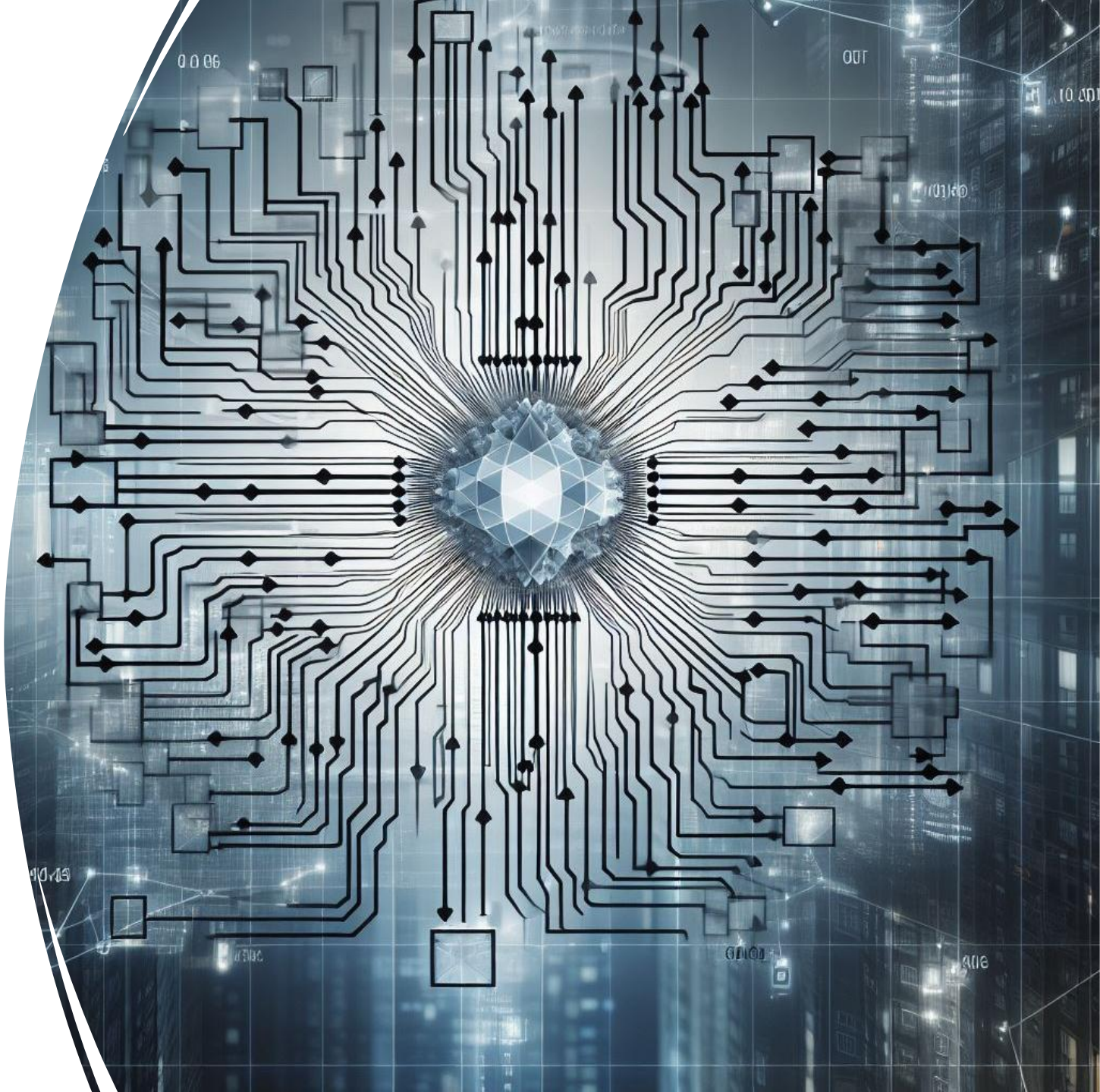
No necesario para dominios simples

Sistemas que requieren consistencia y actualizaciones en tiempo real

Inadecuado para requerimientos de consistencia en tiempo real

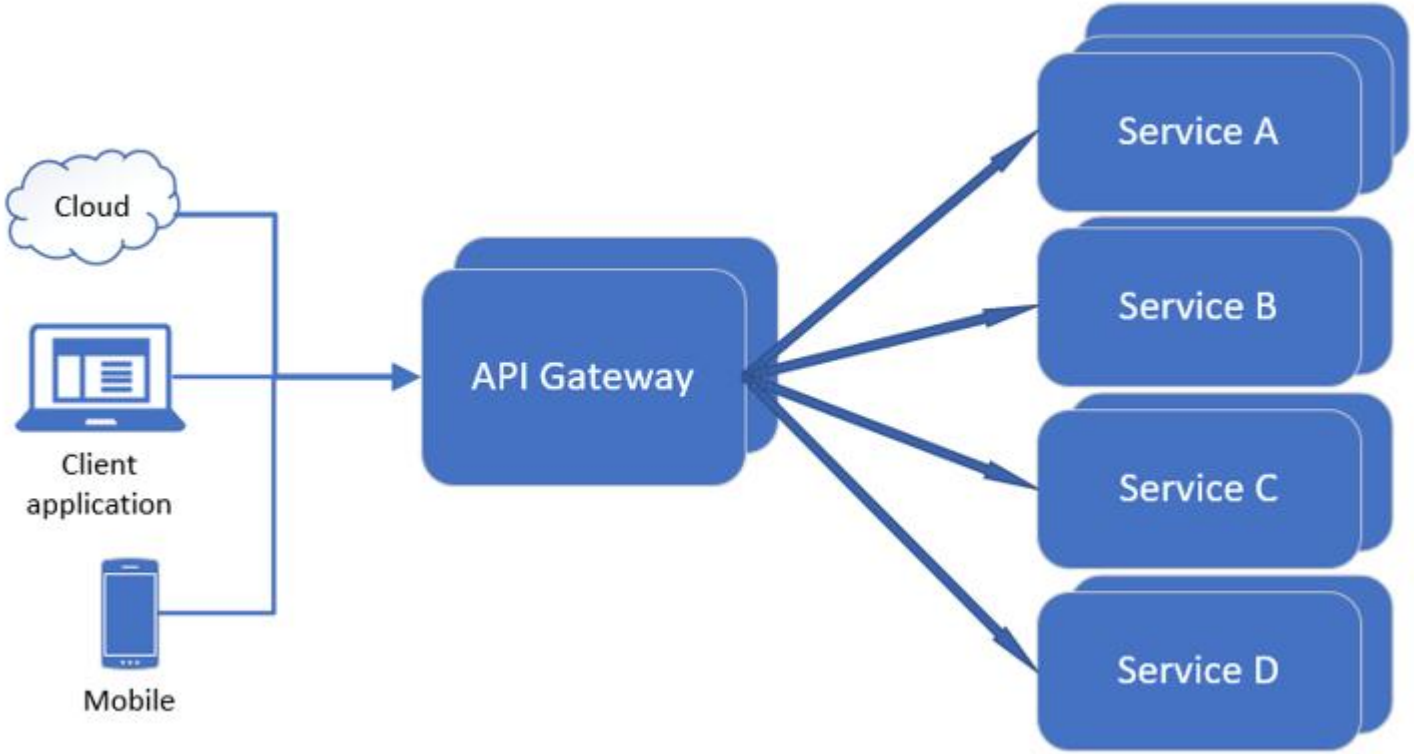


Patrón Gateway Aggregation





Patrón Gateway Aggregation



Gateway Aggregation es un patrón arquitectónico para manejar y optimizar la comunicación entre clientes y múltiples servicios backend mediante un **API Gateway**.

En un sistema distribuido, un cliente puede necesitar datos de varios servicios backend para completar una solicitud.

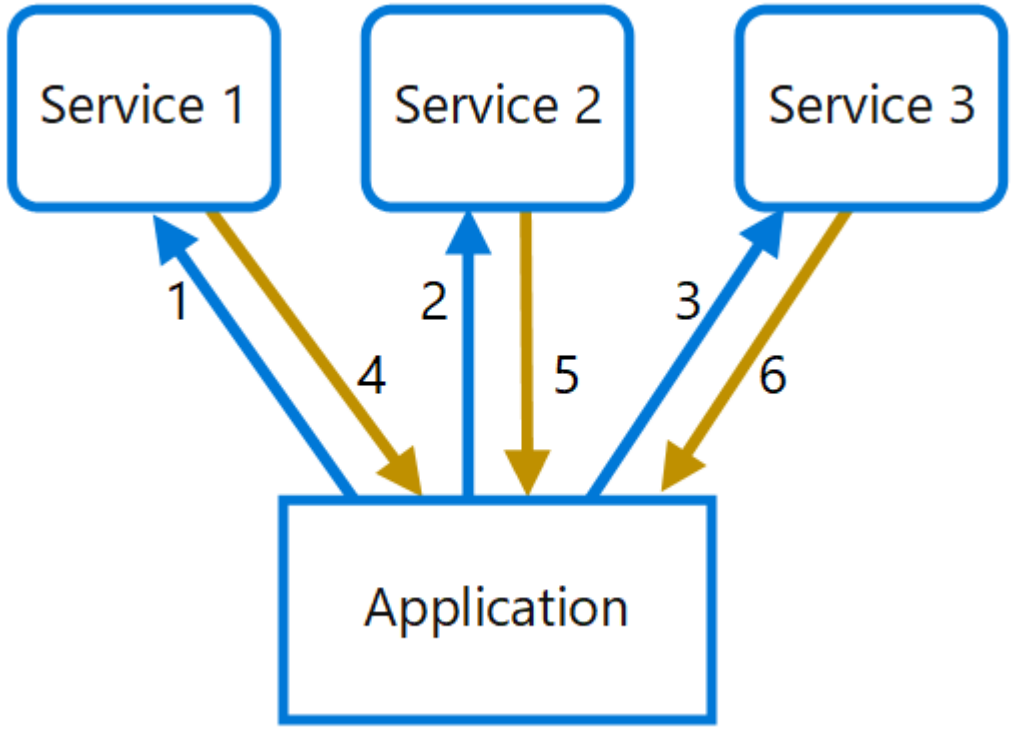


Problema

Múltiples Llamadas a Servicios Backend

Incremento de Recursos y Llamadas de Red

Ineficiencia en Redes de Alta Latencia



Impacto en el Rendimiento y la Escalabilidad

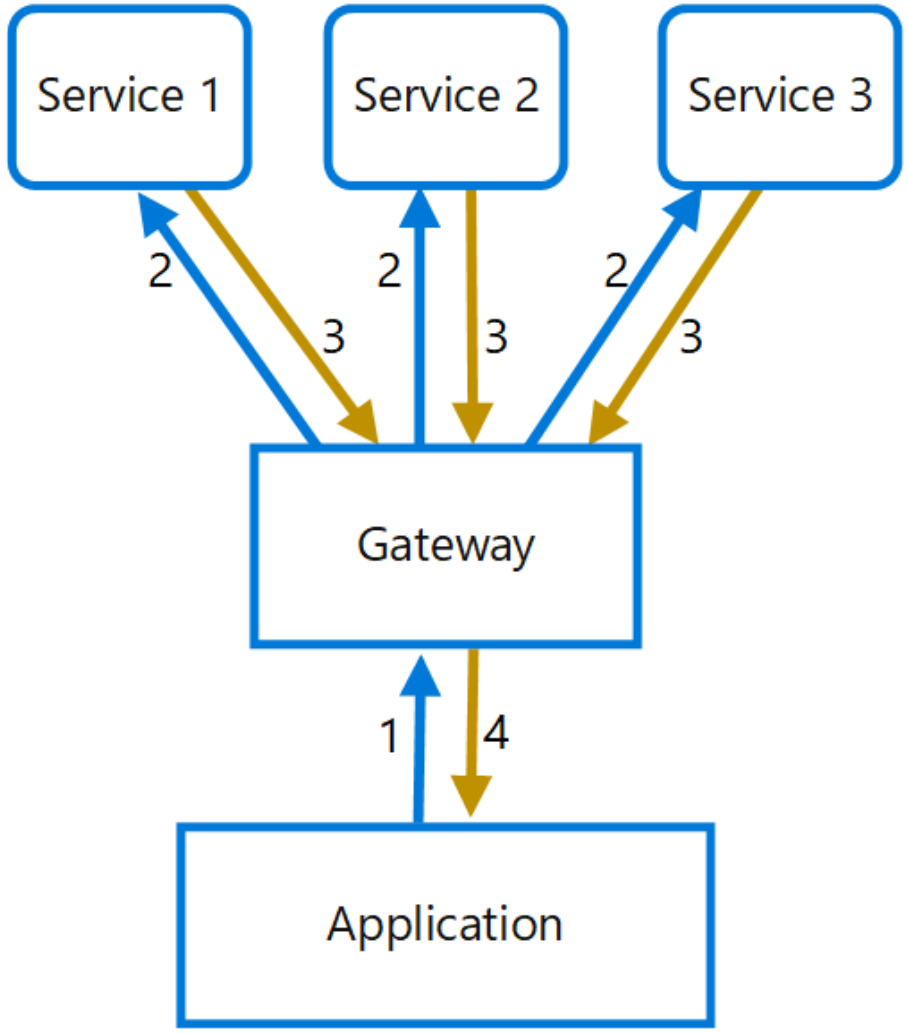
Problemas en Arquitecturas de Microservicios

Paralelismo y Fallos en Conexiones

Solución

Funcionalidad de la Puerta de Enlace

- Recibe solicitudes de los clientes.
- Envía solicitudes a los sistemas de backend.
- Agrega los resultados y los envía de vuelta al cliente.







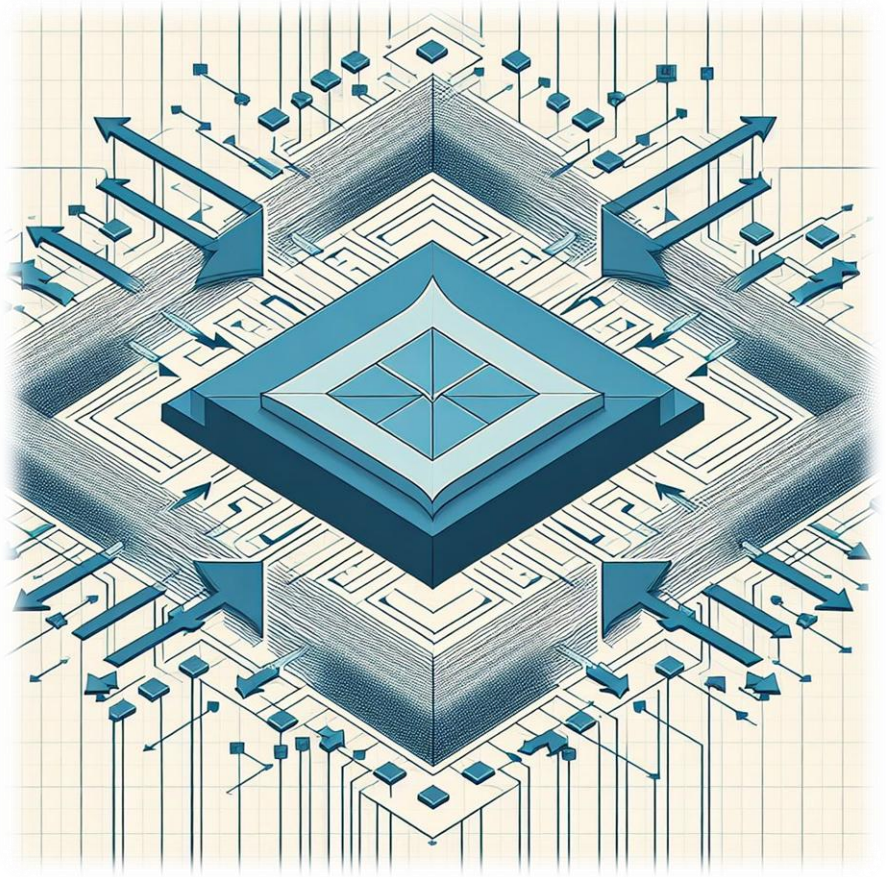
Consideraciones

- Acoplamiento de Servicios
- Ubicación de la Puerta de Enlace
- Punto Único de Falla
- Cuello de Botella
- Pruebas de Carga
- Diseño Resiliente
- Gestión de Retrasos
- E/S Asincrónicas
- Seguimiento Distribuido
- Monitoreo de Métricas
- Datos en Caché
- Servicio de Agregación Dedicado

Ventajas del Event Sourcing

Reducción de la latencia

Simplificación del cliente



Gestión de fallos

Seguridad y autenticación centralizada



¿Cuándo usar CQRS?

Comunicación con Múltiples Servicios de Backend

Cuando un cliente necesita interactuar con varios servicios backend para completar una operación, el patrón de Gateway Aggregation es altamente beneficioso.

Redes de Alta Latencia

Este patrón es ideal para entornos donde las redes tienen una latencia significativa, como en las redes móviles.



¿Cuándo no se recomienda?

Reducción de Llamadas a un Solo Servicio

Si solo necesita reducir la cantidad de llamadas a un único servicio para múltiples operaciones, considere una operación por lotes en lugar de este patrón.

Latencia Insignificante

Cuando la aplicación y los servicios backend están físicamente cerca y la latencia de la red no es un problema significativo, el beneficio de la agregación puede ser mínimo.

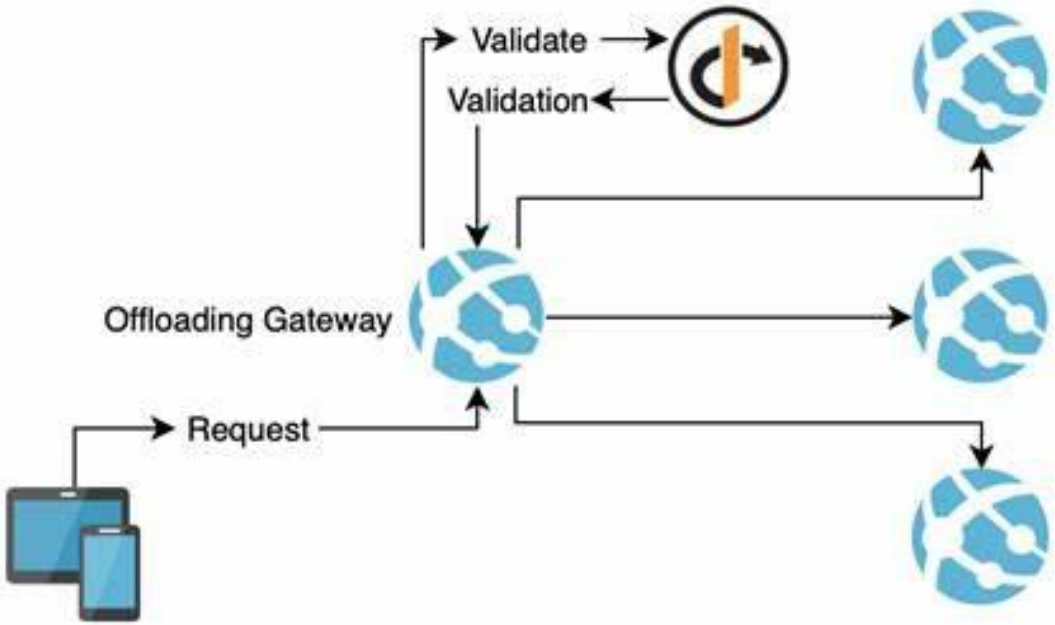


Patrón Gateway Offloading





Patrón Gateway Offloading



El patrón **Gateway Offloading** es una estrategia arquitectónica que se centra en reducir la carga de trabajo de los servicios de backend al centralizar funciones comunes en un Gateway.

Esta Gateway actúa como un intermediario entre el cliente y los servicios de backend, asumiendo responsabilidades como autenticación, autorización, enrutamiento, y cacheo de datos.

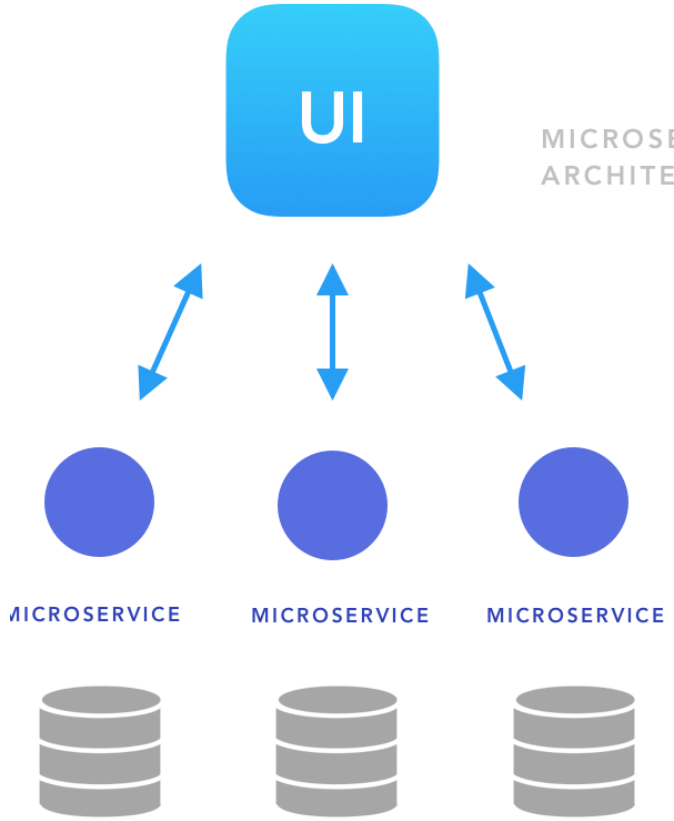


Problema

Múltiples Llamadas a Servicios Backend

Incremento de Recursos y Llamadas de Red

Ineficiencia en Redes de Alta Latencia



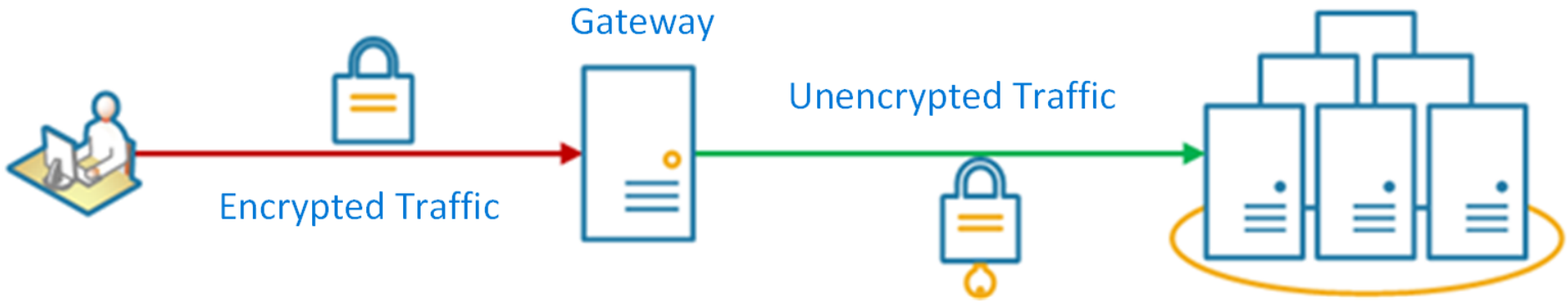
Impacto en el Rendimiento y la Escalabilidad

Problemas en Arquitecturas de Microservicios

Paralelismo y Fallos en Conexiones

Solución

El patrón de Gateway Offloading implica mover la carga de estas responsabilidades comunes a un Gateway que maneja estas funciones de manera centralizada.







Consideraciones

Desacoplamiento y Cohesión

Asegúrese de que la puerta de enlace no introduzca un acoplamiento innecesario entre los servicios de backend.

Latencia

La puerta de enlace debe estar lo más cerca posible de los servicios de backend para minimizar la latencia.

Punto Único de Falla

La puerta de enlace puede convertirse en un único punto de falla si no se diseña adecuadamente.

Cuello de Botella

La puerta de enlace puede convertirse en un cuello de botella si no se escala adecuadamente.

Resiliencia y Gestión de Errores

Implemente estrategias de resiliencia como reintentos, tiempos de espera, y circuit breakers.

Ventajas del Gateway Offloading

Eficiencia y Mantenibilidad



Escalabilidad

Mejora del Rendimiento

Resiliencia



¿Cuándo usar Gateway Offloading?

Necesidad de Funciones Comunes

Utilice este patrón cuando múltiples servicios de backend necesiten manejar funciones comunes como autenticación y autorización.

Alto Volumen de Tráfico

Este patrón es beneficioso en aplicaciones que manejan un alto volumen de tráfico.



¿Cuándo usar Gateway Offloading?

Entornos de Alta Latencia

Utilice este patrón en entornos donde la latencia de la red es una preocupación significativa.

Necesidad de Escalabilidad

Este patrón es adecuado cuando se necesita mejorar la escalabilidad de los servicios.



¿Cuándo no se recomienda?

Bajo Volumen de Tráfico

Puede no ser necesario para aplicaciones con bajo volumen de tráfico.

Latencia No Es un Problema

Si la latencia de la red no es una preocupación, este patrón puede no ofrecer beneficios significativos.



Facultad de
Ingeniería



Universidad de
La Sabana